ABSTRACT
The utilization of machine learning to forecast mental health outcomes presents a viable approach to improving patient care and resource management in the dynamic field of healthcare analytics. The primary objective is to compare how well various models predict Data_Value, a discretized target variable that represents mental health outcomes. This study looks into how different machine learning models can be applied to the "Alzheimer's Disease and Healthy Ageing Data" dataset to predict mental health measures. Finding the best model to predict discretized mental health outcomes that fall under the target variable Data_Value was the main goal. The models used for this research include a variety of methodologies, including tree-based techniques like Decision Trees and Random Forests, which are renowned for their ability to handle complicated datasets with adaptability, and linear models like Logistic Regression, which are prized for their ease of use and interpretability. The SVC, which is well-known for its efficacy in high-dimensional spaces, was also included, as were ensemble techniques like Gradient Boosting, which integrate the predictions of multiple models to increase accuracy. Because of its simplicity and interpretability, the non-parametric KNN model was chosen. Accuracy, precision, recall, and F1 score are used to evaluate each model's performance and give a thorough picture of its predictive power. A detailed evaluation of each model's performance is obtained by utilizing precision, recall, and F1 scores, which take into account the model's potential to reduce false positives as well as accurately identify positive examples. In healthcare settings, where needless interventions can be damaging or expensive, precision which quantifies the percentage of true positive predictions among all positive predictions becomes crucial. To make sure that all cases of interest are found, recall quantifies the percentage of real positives that are appropriately identified. A balanced statistic that takes into consideration both false positives and false negatives, the F1 score harmonic mean of precision and recall offers a thorough evaluation of the efficacy of the model. Using the KBinsDiscretizer, the target variable was discretized into five categories, and missing values were handled as part of the data preprocessing step. To assess the models' generalisability, the dataset is divided into training and testing sets. To predict

important mental health indicators including Alzheimer's disease and frequent anxiety, the project intends to assess how well different machine learning models work. A standardized pipeline framework will be used to create each machine-learning model. This pipeline will contain model fitting and assessment after categorical feature preprocessing using OneHotEncoding. A fair comparison of the models' performance indicators was made possible by the systematic approach that guaranteed consistency among the models. Geographical locations, stratification groups, and demographic data are only a few of the elements included in the dataset used in this study. The study also uses a thorough methodology that begins with data preparation, which involves handling missing values, cleansing the dataset by deleting unnecessary columns and merging year-related data for temporal analysis. An important factor in predicting mental health outcomes is feature selection, which focuses on characteristics including year, class, geography, and stratification groups. The significance of choosing suitable machine learning models based on their interpretability and generalisability in addition to their accuracy is highlighted in this study.

ACKNOWLEDGMENT

I would like to sincerely thank my supervisor [Supervisor's Name] for all of their help and support during my dissertation's writing and for their outstanding advice and helpful feedback. Their expertise and opinions helped me get this assignment done.

I am also very appreciative of the professors and staff members for their consistent assistance and for creating a lively learning atmosphere. My academic journey has been greatly aided by their support and encouragement.

My sincere gratitude goes out to my research partners and colleagues for their friendship, cooperation, and intellectual stimulation. I had an amazing research experience because of their encouragement and the stimulating discussions we had.

Finally, I would like to express my gratitude to everyone who helped me to complete this dissertation, regardless of how trivial the contribution was. I sincerely appreciate all of your assistance and support; it has been really helpful.

Table of Contents

ABSTRACT 0
ACKNOWLEDGMENT 2
TABLE OF FIGURES 4
1 INTRODUCTION 6
1.1 BACKGROUND 7
1.2 RESEARCH AIM AND OBJECTIVES 9
1.3 RESEARCH HYPOTHESIS 9
PRIMARY HYPOTHESIS 9
2 LITERATURE REVIEW 11
2.1 INTRODUCTION 11
2.2 OVERVIEW OF ALZHEIMER'S DISEASE 11
2.3 STAGES OF ALZHEIMER'S 11
2.4 PREVALENCE AND IMPACT 12
2.5 ROLE OF MACHINE LEARNING IN HEALTHCARE 12
2.6 APPLICATION OF MACHINE LEARNING IN HEALTHCARE 13
2.7 ADVANTAGES OF MACHINE LEARNING IN HEALTHCARE 14
3 METHODOLOGY 22
3.1 INTRODUCTION 22
3.2 RESEARCH DESIGN 25
3.3 DATA SOURCE 26
3.4 TOOLS AND TECHNOLOGIES 26
3.5 ETHICAL CONSIDERATION 27
3.6 RESEARCH LIMITATIONS 28
4 RESULTS AND IMPLEMENTATION 28
4.1 Data Preprocessing 28
4.2Random Forest Classifier 30
4.3 SUPPORT VECTOR CLASSIFIER 35
4.4 GRADIENT BOOSTING CLASSIFIER 40
4.5 LOGISTIC REGRESSION 44
4.6 DECISION TREE CLASSIFIER 48
4.7 KNN CLASSIFIER 52
4.8 COMPARISON OF Accuracy, F1 Score, Recall and Precision 57
4.8.1 ACCURACY 57
4.9 SUMMARY TABLE 60
5 PROJECT MANAGEMENT 61
5.1 PROJECT PLANNING 61
5.2 RISK MANAGEMENT 61
6 DISCUSSION 63

7 CONCLUSION 64
7.1 ACHIEVEMENTS 64
7.2 FUTURE WORKS 65
REFERENCES 66
APPENDIX : A 71
CODE AND OUTPUT 71
Appendix: B ..... **Error! Bookmark not defined.**
Certificate of Ethical Approval ..... **Error! Bookmark not defined.**

TABLE OF FIGURES

Figure 1:Machine Learning Model Training Workflow 7
Figure 2: Hypothesis Network for Machine Learning in Mental Health Prediction 10
Figure 3: Alzheimer's with aging 12
Figure 4:Machine Learning in Healthcare Visualization 12
Figure 5: Machine Learning in Healthcare Workflow 14
Figure 6: EEG Electrode Placement 16
Figure 7:Brain MRI Scans 19
Figure 8:Risk Factors of Alzheimer's Disease 20
Figure 9: Alzheimer’s Disease Workflow 26
Figure 10:Data Preprocessing 29
Figure 11:Data Summary 30
Figure 12:Random Forest Classifier 30
Figure 13:Encoding Categorical Features 32
Figure 14: Classification Report for RFC 33
Figure 15:Actual vs Prediction plot for RFC 34
Figure 16:Confusion Matrix for RFC 34
Figure 17::Support Vector Classifier Model Setup 35
Figure 18: Encoding Categorical for Support Vector Classifier 36
Figure 19:Classification Report of SVM 37
Figure 20: Actual vs Predicted plot for SVM 38
Figure 21:Confusion Matrix Of SVM 39
Figure 22: Gradient Boosting Classifier Setup 40
Figure 23:Encoding Categorical for GBM 41
Figure 24:Classification Report of GBM 41
Figure 25:Actual vs Prediction plot of GBM 42
Figure 26:Confusion Matrix of GBM 43
Figure 27:Logistic Regression Model Setup 44
Figure 28:Encoding Categorical for Logistic Regression 45
Figure 29:Classification Report of Logistic Regression 46
Figure 30:Confusion Matrix of Logistic Regression 47
Figure 31:Decision Tree Classifier model setup 48
Figure 32:Encoding Categorical for Decision Tree Classifier 49
Figure 33:Classification Report of Decision Tree Classifier 50
Figure 34:Actual vs Prediction of Decision Tree Classifier 50
Figure 35: Confusion Matrix of Decision Tree Classifier 51
Figure 36:KNN model Setup 52

Figure 37:Encoding Categorical of KNN..... 53
Figure 38:Classification Report of KNN..... 54
Figure 39:Actual vs Prediction of KNN 55
Figure 40:Confusion Matrix of KNN..... 56
Figure 41:Model Comparison Accuracy 57
Figure 42:Model Metrics Comparison..... 58
Figure 43:Model Performance Summary Table..... 60

1 INTRODUCTION

The potential for predictive insights has been transformed by the incorporation of machine learning into healthcare analytics, especially in the area of mental health. The prevalence of mental health problems, including those linked to Alzheimer's disease and general cognitive loss, is a major worry as societies get older. Predictability with accuracy and prompt action are essential for enhancing patient outcomes and making the most use of healthcare resources. This study uses data from the "Alzheimer's Disease and Healthy Ageing Data" dataset to investigate the use of machine learning algorithms to predict mental health measures.

The dataset includes a wide range of variables that are useful for comprehending and forecasting mental health outcomes, such as demographic data, stratification categories, and geography data. These characteristics are essential because they provide the data context and richness, which enables models to identify intricate linkages and patterns that could have an impact on mental health. For example, geography, gender, and age all have a big influence on how common and severe mental health problems are, which makes them crucial variables in predictive modeling. Random Forest, Support Vector Classifier (SVC), Gradient Boosting, Logistic Regression, Decision Tree, and K-Nearest Neighbours (KNN) are the six machine learning methods that were thoroughly evaluated in this survey. An extensive picture of each model's predictive power was obtained by evaluating its performance in terms of accuracy, precision, recall, and F1 score. Cleaning and transforming the dataset to make it acceptable for model training is known as data preprocessing, and it is an important stage in the research process. This involves encoding categorical variables a must for algorithms that need numerical input and handling missing values, which, if not properly handled, might distort the results. Separated into categories is Data_Value, the target variable of the dataset that represents measurements related to mental health. Because outcomes in practical applications are frequently organized into discrete categories rather than continuous variables, this discretization aids in aligning the predictive modeling work with those applications. Additionally, the study highlights how crucial it is to divide the dataset into training and testing sets. To evaluate the model's capacity to generalize to new, untested data and provide an accurate assessment of its predicted performance, this procedure is essential. To ensure that the assessment measures accurately reflect the model's efficacy in real-world circumstances, the testing set is held aside for

performance evaluation after the models have been fitted using the training set. The study aims to identify the optimal strategy for precise and trustworthy predictions by analyzing and contrasting several machine-learning approaches. Through better decision-making and budget allocation, the findings should help legislators and healthcare professionals provide better care for their patients.

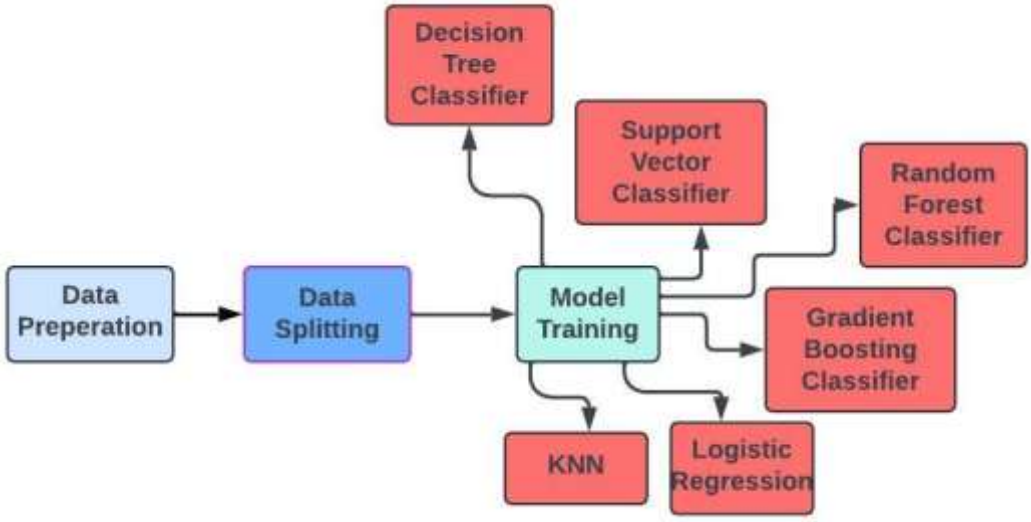


Figure 1: Machine Learning Model Training Workflow

1.1 BACKGROUND

Significant progress has been made in the field of predictive analytics, namely as a result of the convergence of technology and healthcare. Predicting and treating mental health issues, which are becoming more common in older populations, is one area in which this is particularly relevant. Mental health issues, such as those associated with Alzheimer's disease and general cognitive decline, present significant difficulties to global healthcare systems as well as to individuals and their families. Accurately predicting these disorders can result in better management techniques, earlier treatments, and better patient outcomes. The "Alzheimer's Disease and Healthy Ageing Data" collection offers a comprehensive information source suitable for predictive modeling applications. Numerous elements are included in this dataset, including geographic data, stratification categories, and demographic information. These characteristics are important because they include a broad variety of variables, such as age, gender, socioeconomic level, and

accessibility to healthcare in one's area, that might affect mental health outcomes. Because of its richness, this data can be analyzed in a way that may reveal patterns that are not immediately obvious. Machine learning has become an important tool in the field of healthcare analytics in recent years. Machine learning models are capable of handling big datasets with intricate, non-linear correlations between variables, which is not possible with standard statistical methods. This capacity is especially helpful for forecasting outcomes related to mental health, as the process might become more complex due to several interacting components. Different techniques for classification tasks are known to be taken by machine learning models, including Random Forest, Support Vector Classifier (SVC), Gradient Boosting, Logistic Regression, Decision Tree, and K-Nearest Neighbours (KNN). These models are perfect for the kind of intricate research needed to generate predictions about mental health because they can learn from data to spot trends and anticipate outcomes. These models are assessed using criteria other than basic accuracy. The expense of false positives misclassifying a healthy person as unwell and false negatives failing to detect an ill person in the healthcare system can be high. As a result, parameters like recall, precision, and F1 score are also taken into account. The F1 score offers a balance between recall and precision. The accuracy of the positive predictions is measured by precision, while the ability to catch all real positive cases is evaluated by recall. In situations where there may be an imbalance in the distribution of classes, these metrics provide a more nuanced assessment of the model's performance. The goal of this study is to identify the best machine learning model for predicting mental health outcomes, offering knowledge that could be very useful to politicians and healthcare professionals. The study aims to provide insights into the strengths and limits of various models to facilitate the creation of forecasting tools that are more dependable and accurate.

1.2 RESEARCH AIM AND OBJECTIVES

AIM:

This study uses the “Alzheimer’s Disease and Healthy Ageing Data” dataset to evaluate and compare the effectiveness of different machine learning models in predicting mental health metrics in older populations. Finding the most accurate and dependable model to predict mental health outcomes can help with early diagnosis and intervention measures, enhancing patient care and healthcare resource management.

OBJECTIVES :

- Clean up, transform, handle missing values, eliminate extra columns, combine data about a certain year, and encode categorical variables as part of the preprocessing step of the dataset.
- Determine which relevant features such as year, stratification groups, class, and location have an impact on mental health measures.
- Several machine learning models must be developed and trained, including Random Forest, Gradient Boosting Classifier, Decision Tree, Logistic Regression, Support Vector Classifier (SVC), and K-Nearest Neighbours (KNN).
- Analyse each model’s performance using F1 score, accuracy, precision, and recall metrics.
- To verify the accuracy and dependability of the algorithms, forecast mental health measures using fresh, untested data samples.

1.3 RESEARCH HYPOTHESIS

PRIMARY HYPOTHESIS

The “Alzheimer’s Disease and Healthy Ageing Data” dataset can be used to train machine learning models that accurately and precisely predict mental health metrics with high recall, accuracy, and F1-score in aging populations.

SECONDARY HYPOTHESIS

- Machine learning models perform far better in predicting mental health measures when the data has undergone preprocessing, which includes cleaning, converting, and encoding categorical variables.
- Model predictive power is increased through feature selection that takes into account year, stratification categories, class, and location.
- In terms of forecasting mental health measures, several machine learning models including Support Vector Classifier, Gradient Boosting Classifier, Random Forest Classifier, Logistic Regression, Decision Tree Classifier, and KNN Classifier show various advantages and disadvantages.
- Classification performance of the machine learning models is improved by discretizing the target variable Data_Value into appropriate categories.
- Healthcare providers and policymakers can benefit from the insights gained from these models in creating focused interventions and allocating resources for mental health care in older populations as best they can.
- These models' applicability and predictive accuracy for mental health prediction can be further improved by future machine learning advancements and superior approaches.

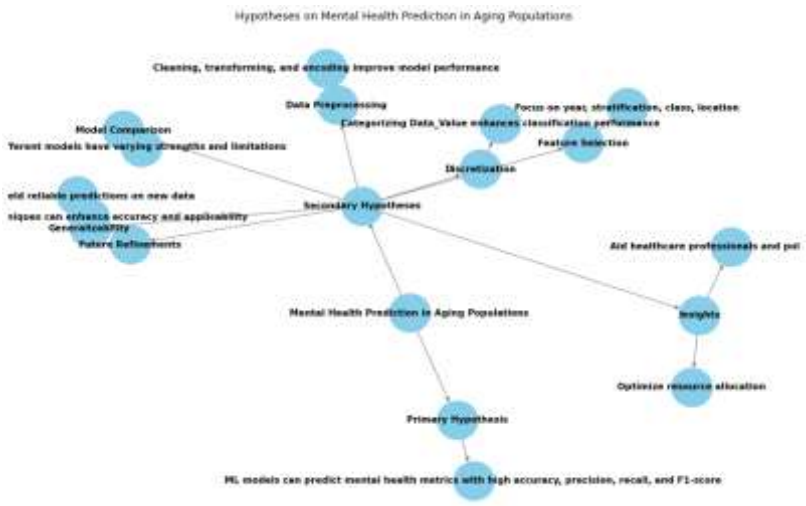


Figure 2: Hypothesis Network for Machine Learning in Mental Health Prediction

2 LITERATURE REVIEW

2.1 INTRODUCTION

The importance of mental health difficulties among aging populations is growing due to the world's growing proportion of elderly people. Conditions including dementia, Alzheimer's disease, anxiety, and depression are more common in the elderly and can have a detrimental impact on their quality of life as well as cause issues for healthcare systems. Traditional methods for assessing mental health frequently involve clinical exams and subjective assessments, but they can be time-consuming and unreliable. Many machine learning models have been effectively used to forecast health outcomes, including Support Vector Machines, Random Forest, Gradient Boosting, Logistic Regression, Decision Tree Classifier, and KNN Classifier.

2.2 OVERVIEW OF ALZHEIMER'S DISEASE

Alzheimer's disease accounts for 60–80% of cases, it is the most frequent cause of dementia. It is defined by the build-up of neurofibrillary tangles and amyloid plaques in the brain, which causes atrophy and damage to the neurons. Clinical evaluations, neuropsychological testing, neuroimaging, and biomarkers like tau and amyloid levels in cerebrospinal fluid (CSF) have historically been used to make diagnoses. Even with the advancements in these diagnostic techniques, early and more precise prediction models are still required for the identification of high-risk people and the tracking of disease progression.

2.3 STAGES OF ALZHEIMER'S

Mild Alzheimer's disease: Memory loss, trouble recalling recent events, and difficulty with routine tasks are among the early signs.

Moderate Alzheimer's disease: There is still a loss in cognitive and functional abilities. Patients may become more confused, have trouble identifying friends and family, and exhibit behavioral abnormalities.

Severe Alzheimer's disease: May become non-responsive to their surroundings, become non-communicative, and need 24-hour care.

2.4 PREVALENCE AND IMPACT

Over 50 million individuals worldwide suffer from Alzheimer's disease, and as the population ages, it is predicted that this figure will triple by 2050. It is predicted that by 2030, the financial burden from care expenses and lost productivity will exceed $1 trillion. Carers experience severe effects as well, including financial, physical, and emotional strain. These figures demonstrate how urgently accurate prediction and early diagnosis are needed.

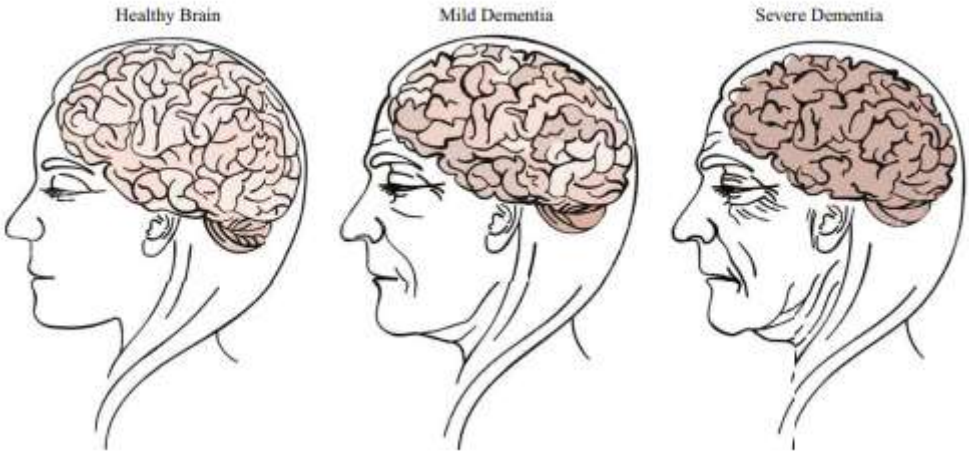


Figure 3: Alzheimer's with aging

2.5 ROLE OF MACHINE LEARNING IN HEALTHCARE

Healthcare has been transformed by machine learning (ML), a branch of artificial intelligence. Its numerous medical applications benefit greatly from its capacity to process enormous volumes of data, identify intricate patterns, and forecast the course of diseases. It also allows for the customization of treatment regimens. With an emphasis on its uses, advantages, and difficulties, this section examines the revolutionary role that machine learning (ML) is playing in the healthcare industry.

Figure 4:Machine Learning in Healthcare Visualization

2.6 APPLICATION OF MACHINE LEARNING IN HEALTHCARE

1. **Prognosis and Diagnosis of Disease:**

   Early Detection: ML models can examine medical imaging data to identify diseases such as cancer early on, which can greatly enhance patient outcomes. One example is the great accuracy with which convolutional neural networks (CNNs) have been able to identify breast cancer from mammograms.

   Predictive Analytics: ML algorithms can analyze patient data, including genetic information, lifestyle factors, and clinical history, to predict the development and progression of chronic diseases, such as diabetes and cardiovascular diseases.

2. **Personalised Health Care:**

   Treatment Optimisation: Based on a patient's genetic composition and medical background, ML models can forecast how they will react to different therapies, enabling individualized therapeutic approaches. This is especially helpful in oncology since tumor genetics can be used to create individualized therapy regimens.

   Drug Discovery: By predicting molecular behavior, identifying promising drug candidates, and refining clinical trial designs, machine learning (ML) expedites the process of finding new drugs.

3. **Imaging and Diagnostics in Medicine:**

   Image Analysis: By using machine learning approaches, medical image analysis can be improved in terms of abnormality detection and categorization. For instance, in radiology, ML algorithms can discern between benign and malignant lesions.

   Automated Diagnostics: By automating the detection of anomalies, machine learning models help radiologists and pathologists save time and increase the precision of their diagnoses.

4. **Patient Surveillance and Handling:**

   Remote Monitoring: Wearable technology and Internet of Things (IoT) devices gather real-time health data, which machine learning (ML) algorithms evaluate to track patient conditions and anticipate any problems before they get serious.

   Chronic Condition Management: By evaluating patient data in real-time,

offering insights into the course of the condition, and recommending treatments, machine learning models assist in the management of chronic illnesses.

2.7 ADVANTAGES OF MACHINE LEARNING IN HEALTHCARE

1. **Increased Accuracy:** ML algorithms can spot correlations and patterns that human clinicians miss, which results in diagnoses and forecasts that are more accurate.
2. **Efficiency:** Healthcare workers may concentrate on more difficult cases and patient care when regular chores and analysis are automated.
3. **Personalisation:** Machine learning (ML) helps create more focused and successful healthcare interventions by customizing treatment plans based on patient data.
4. **Cost reduction:** Healthcare systems can save a lot of money by implementing effective treatment strategies, early disease detection, and improved diagnostic

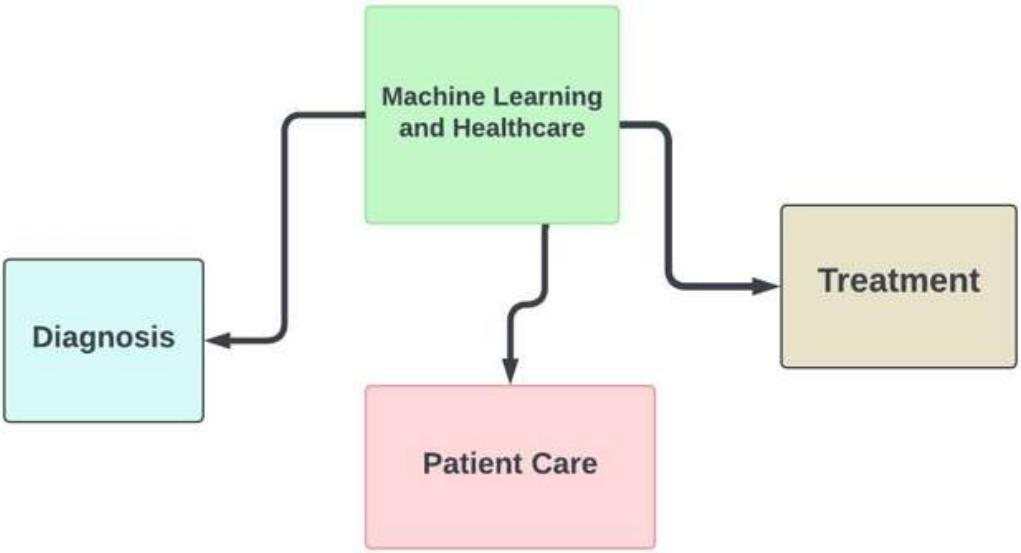


Figure 5: Machine Learning in Healthcare Workflow

2.8 CASE STUDIES

Machine learning techniques for the diagnosis of Alzheimer's disease: A review

The application of machine learning (ML) in the diagnosis of Alzheimer's disease (AD) is thoroughly examined in the study "Machine Learning Techniques for the Diagnosis of Alzheimer's Disease: A Review," written by M. Tanveer and colleagues. Various machine learning (ML) methods, including support vector machines (SVM), neural networks, and deep learning, are categorized in this review with an emphasis on how they might be applied to medical data, including biomarkers, neuroimaging, and clinical data. Preprocessing methods like feature extraction and normalization are critical for enhancing the efficacy and precision of machine learning models, and the authors stress their significance.

The article examines many machine learning (ML) approaches to Alzheimer's disease diagnosis, emphasizing the benefits of deep learning for feature extraction and support vector machines (SVMs) for managing high-dimensional data. It mentions difficulties that include the requirement for substantial computer power and datasets, especially for deep learning. The review also underlines how crucial it is for models to be interpretable for clinical application, and it makes recommendations for future studies to improve the generalisability of machine learning methods across a range of demographics, incorporate multi-modal data, and create more interpretable models. These developments may lead to better patient outcomes and early diagnosis.

Improving Alzheimer's Disease Diagnosis with Machine Learning Techniques

The study "Improving Alzheimer's Disease Diagnosis with Machine Learning Techniques" explores how several machine learning (ML) approaches may improve the identification of Alzheimer's disease (AD). The application of ensemble methods, neural networks, and support vector machines (SVMs) is highlighted, along with how well they handle complicated, high-dimensional data, such as genetic markers and neuroimaging. To increase the precision and efficacy of machine learning models, the paper emphasizes the significance of preprocessing techniques like dimensionality reduction and normalization. To find the most pertinent biomarkers for AD and lower computational costs while enhancing model interpretability, feature selection is also emphasized as a crucial step in the process.

The study also tackles important issues facing the industry, such as the lack of big, labeled datasets needed to train reliable machine learning models and the crucial problem of model interpretability, which is vital to winning over medical experts. It promotes multi-modal data integration, which combines clinical, genomic, and neuroimaging data to offer a more thorough and precise diagnostic framework. To translate these advances in machine learning into useful therapeutic tools that could help patients with Alzheimer's disease receive better care and diagnoses earlier, the authors urge greater study into the development of standardized datasets and more interpretable models.

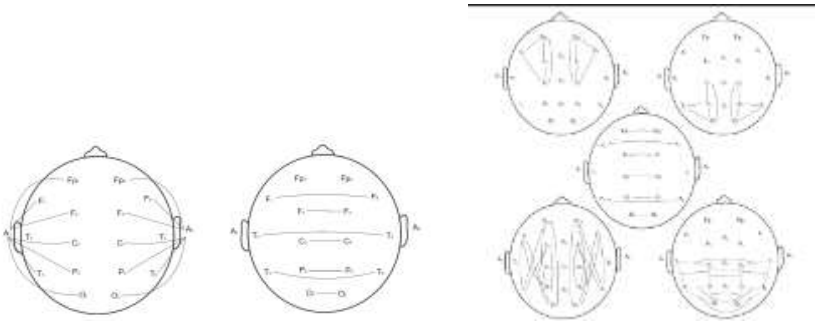


Figure 6: EEG Electrode Placement

Classification of Alzheimer’s Disease Using Machine Learning Techniques

The application of machine learning (ML) and data mining techniques for the classification of Alzheimer's Disease (AD) stages is explored in the study "Classification of Alzheimer's Disease using Machine Learning Techniques" by Muhammad Shahbaz, Shahzad Ali, Aziz Guergachi, Aneeta Niazi, and Amina Umer. This study compares six algorithms: k-nearest neighbors, decision tree, rule induction, Naive Bayes, generalized linear model (GLM), and deep learning using the Alzheimer's Disease Neuroimaging Initiative (ADNI) dataset. In identifying the stages of AD, the GLM had the highest accuracy, at 88.24%. The age of the patient, whole brain volume, and the Clinical Dementia Rating Sum of Boxes (CDRSB) test are found to be the main differentiating factors.

The research highlights the promise of machine learning approaches in the early diagnosis and detection of Alzheimer's disease (AD). It recommends further refining these algorithms by increasing the number of data examples, especially for under-represented phases such as Subjective Memory Complaint (SMC) and Early Mild

Cognitive Impairment (EMCI). To accurately classify Alzheimer's disease phases, the authors emphasize the significance of differentiating characteristics such as brain volume measurements and results on cognitive tests. This study demonstrates how well machine learning (ML) works to improve patient care and diagnostic accuracy in the healthcare industry.

Early Diagnosis of Alzheimer's disease using Machine Learning Techniques: A Review

Machine learning (ML) techniques for the early detection of Alzheimer's disease (AD) are thoroughly reviewed in the study "Early Diagnosis of Alzheimer's Disease using Machine Learning Techniques: A Review" by Aunsia Khan and Muhammad Usman. ML techniques such as association rule mining, neural networks, and support vector machines are discussed in the study, focusing on how they might be used to handle and analyze complex medical data to identify AD early on. Managing big datasets, extracting pertinent features, and enhancing diagnostic precision are three areas in which these methods shine.

The authors point out important obstacles facing current research on Alzheimer's disease (AD), such as the reliance on unreliable datasets and class imbalance that may compromise diagnostic accuracy. They provide a model that includes meticulous feature selection to improve model performance and preprocessing measures to address class imbalance. By enhancing early AD diagnosis accuracy and reliability, this strategy may result in better patient outcomes via prompt intervention and enhanced disease management. The article highlights how machine learning can revolutionize the diagnosis of AD, particularly in its early stages.

Alzheimer's Disease Early Detection Using Machine Learning Techniques

The aim of the study "Alzheimer's Disease Early Detection Using Machine Learning Techniques" is to use machine learning (ML) to create a computer-aided diagnosis system for the early detection of Alzheimer's disease (AD). Two datasets are used in the study: the Open Access Series of Imaging Studies (OASIS) and the Alzheimer's Disease Neuroimaging Initiative (ADNI). To categorize patients as cognitively normal or having AD, it uses a variety of machine learning approaches, including logistic regression, support vector machines, random forests, and others. For example, the models' accuracy rates were quite high in the ADNI dataset, with 99.43% and 99.10% accuracy rates for logistic regression and support vector machines,

respectively. As prompt management could lead to better patient outcomes, the paper emphasizes the critical role that early identification plays in controlling AD. According to the research, machine learning (ML) approaches can greatly help with early AD detection, providing a potential tool for medical personnel to detect the disease before it advances to more severe stages.

Early-stage Alzheimer’s disease prediction using Machine Learning Models

The study "Early-Stage Alzheimer's Disease Prediction Using Machine Learning Models" looks at the effectiveness of machine learning methods for early Alzheimer's Disease (AD) prediction. Employing data from the Open Access Series of Imaging Studies (OASIS), the study uses several classifiers, including Decision Tree, Random Forest, Support Vector Machine (SVM), Gradient Boosting, and Voting classifiers. Metrics including recall, accuracy, precision, and F1-score are used to assess these models. For improved management and treatment outcomes in AD, the study emphasizes the significance of early detection. The validation accuracy of 83% achieved by the suggested methods is higher than that of many other approaches, demonstrating a notable improvement in prediction accuracy. This breakthrough emphasizes machine learning's potential for diagnosing illnesses, especially Alzheimer's disease, in medicine.

A Novel Approach Utilizing Machine Learning for the Early Diagnosis of Alzheimer's Disease

The study on the use of machine learning (ML) techniques for the early diagnosis of Alzheimer's disease (AD) is presented in the publication "A Novel Approach Utilising Machine Learning for the Early Diagnosis of Alzheimer's Disease" which was published in Biomedical Materials & Devices (2023). Classifiers such as Gaussian Naive Bayes, Decision Tree, Random Forest, XGBoost, Gradient Boost, and Voting classifiers are used in the study, which also uses the OASIS dataset. ML can improve early AD identification, as evidenced by the study's greatest accuracy of 96% with the Voting classifier. For improved disease treatment and patient outcomes, this early diagnosis is essential. To increase the predicted accuracy of the model, the study also highlights how important it is to choose features, handle missing data, and properly preprocess data.

The study adds to the expanding field of healthcare analytics by demonstrating the useful insights that sophisticated machine learning algorithms can offer and how they can help with the early detection of neurodegenerative illnesses like Alzheimer's disease.

A review of the application of Deep learning in the detection of Alzheimer's disease

To identify Alzheimer's disease (AD) in its early stages, convolutional neural networks (CNNs) are investigated in the publication "A Review of the Application of Deep Learning in the Detection of Alzheimer's Disease." The study employs two main techniques: transfer learning with the pre-trained neural network VGG19 model and basic 2D and 3D CNN architectures. Different stages of AD, such as normal control, early mild cognitive impairment (EMCI), late mild cognitive impairment (LMCI), and AD itself, were categorized using these methodologies. With the VGG19 model achieving 97% accuracy in multi-class classification, the study demonstrated remarkable accuracy. The research also includes a web application that enables remote AD diagnosis and staging, offering patients and healthcare providers a useful tool, particularly in situations like the covid 19

This tool could help patients by facilitating prompt AD management and intervention. Improved healthcare strategies for controlling AD can be achieved by utilizing advanced deep learning techniques, as demonstrated by this study, which can considerably improve the accuracy and efficiency of AD diagnosis.

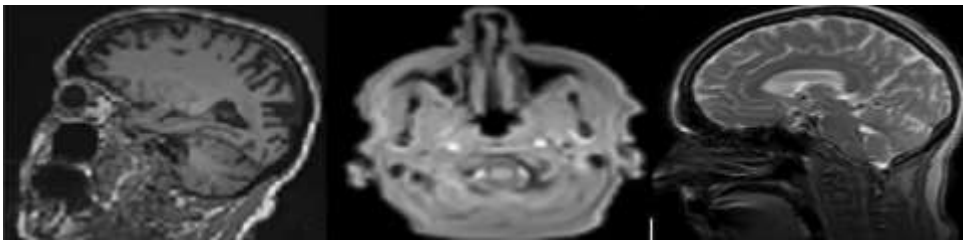


Figure 7:Brain MRI Scans

Alzheimer's Disease Diagnosis Using Machine Learning: A Survey

A detailed overview of the application of machine learning (ML) techniques for Alzheimer's disease (AD) diagnosis can be found in the publication "Alzheimer's Disease Diagnosis Using Machine Learning: A Survey," which was published in

Applied Sciences. With more than 80 publications reviewed since 2017, it assesses different machine learning models such as ensemble approaches, decision trees, and support vector machines and highlights how well they work for detecting AD by analyzing neuroimaging and non-image biomarkers. Key difficulties include data quality, model interpretability, and the need for more effective early detection strategies are identified by the study, which divides research into three categories: data-related, methodology-related, and medical-fostering. The findings highlight the significance of taking lifestyle, environmental, and genetic factors into account while studying AD. The paper ends by outlining potential avenues for future study to improve early intervention tactics and diagnostic accuracy, emphasizing the contribution of cutting-edge machine learning techniques to better patient outcomes and medical procedures.

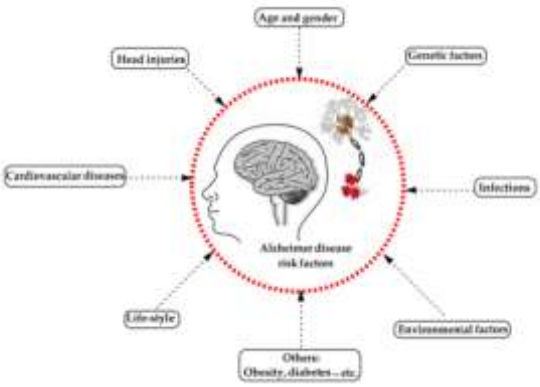


Figure 8:Risk Factors of Alzheimer's Disease

Alzheimer's Disease diagnosis and classification using Deep Learning

Alzheimer's disease (AD) diagnosis and classification using deep learning models DenseNet-169 and ResNet-50 CNN architectures, in particular, is investigated in the study "Alzheimer's Disease Diagnosis and Classification Using Deep Learning Techniques" by Waleed Al Shehri. DenseNet-169 outperforms ResNet-50 in training accuracy, allowing the study to classify AD into four categories: non-dementia, very mild dementia, mild dementia, and moderate dementia. Because manual diagnosis of AD is difficult due to the large number of cases and the nuanced nature of early symptoms, the research shows how deep learning can be used to automate and improve AD diagnosis accuracy.

The work makes use of MRI scans from an open-source dataset, which has been enhanced in quantity and quality by preprocessing methods such as data

augmentation. Metrics such as accuracy are used to assess the models, and the findings are encouraging when it comes to real-time use in medical diagnostics. The report ends with recommendations for further research, such as growing the dataset and improving the diagnostic algorithms to improve their accuracy and early detection capabilities.

3 METHODOLOGY

3.1 INTRODUCTION

The "Alzheimer's Disease and Healthy Ageing Data" dataset is used in this study's technique to systematically assess the predictive ability of different machine learning models in predicting mental health metrics. The procedures for preprocessing data, choosing features, training and evaluating machine learning models, and determining model performance indicators are all covered in this part.

The process also includes feature selection, which is centered on locating and employing the most pertinent factors that have a major influence on mental health projections, such as demographic data, stratification categories, and geographic information. This increases the predicted accuracy of the models by ensuring that they are trained on the most illuminating parts of the data.

Many machine learning models, such as Decision Tree, Gradient Boosting, Random Forest, Support Vector Classifier (SVC), Logistic Regression, and K-Nearest Neighbours (KNN), are used. These models were selected to provide a thorough evaluation of their advantages and disadvantages because of their varied methods for processing data and generating predictions.

Important measures including accuracy, precision, recall, and F1-score are used to thoroughly assess the models' performances. To determine which model is best for real-world use, the research also includes a comparison analysis. The models are evaluated on fresh, untested data samples to show their practicality. The methodology also identifies possible avenues for future research, with an emphasis on exploring cutting-edge methodologies to further enhance prediction skills and refine existing models. In the end, this methodical approach hopes to improve healthcare outcomes for the elderly population by offering insightful information about the application of machine learning in mental health prediction.

TECHNIQUES:

Several machine learning models were utilized to forecast the discretized mental health metrics. These models were selected based on their differences in data processing and prediction techniques, which allowed for a thorough assessment of their advantages and disadvantages.

The following are the techniques used :

1. Random Forest Classifier

2. Support Vector Classifier
3. Gradient Boosting Classifier
4. Logistic Regression
5. Decision Tree Classifier
6. KNN

Random Forest Classifier: combines the outputs of several decision trees built during training to enhance prediction accuracy and manage overfitting. This approach is known as ensemble learning. A Random Forest is a voting system where each tree casts a vote to determine the winner, with the most popular class being chosen. Complex classification problems might benefit from this model's great robustness and ability to handle huge datasets with a multitude of features.

Support Vector Classifier: is a type of Support Vector Machine that is used for classification tasks; it finds the hyperplane in the feature space that best divides the classes; it is useful in high-dimensional spaces, especially for datasets where the number of features is greater than the number of samples; it can create decision boundaries even in non-linear spaces, which makes it a powerful tool for complex classification problems.

Gradient Boosting Classifier: is a group method that creates models one after the other, trying to fix the mistakes of the earlier models. By concentrating on examples that are challenging to forecast, this strategy, referred to as boosting, helps the model perform better. Because of its excellent accuracy, gradient boosting is frequently employed in competitive machine learning applications.

Logistic Regression: is a statistical model that is frequently applied to tasks involving binary and multiclass classification. By using the logistic function on a linear combination of the input features, it calculates the likelihood that a given input is a member of a particular class. For classification tasks, the model is helpful because it produces probabilities that can be mapped to categorical outcomes. Since it offers a clear understanding of the correlations between characteristics and outcomes, logistic regression is highly praised for its simplicity and interpretability.

Decision Tree Classifier: is a model that classifies data at each node by dividing it into subsets according to feature values. Because the tree structure resembles how people make decisions, it is simple to see and understand. Decision trees are helpful because they are simple and interpretable and can handle numerical and categorical

data.

KNN: is a classification algorithm that is instance-based and non-parametric. It uses the categories of a new data point's closest neighbors in the feature space to forecast the category of that data point. Because KNN uses the majority class among the neighbors to classify data, it is easy to build and comprehend. It is sensitive to irrelevant information and the distance metric selected, though, and can be computationally costly for large datasets.

DATA PREPROCESSING

To assure the quality and utility of the dataset, data preparation is an essential stage in any machine learning project. Several preprocessing methods were used in this study, including:

Managing Missing Values: In real-world datasets, missing data is a regular problem. The study included various techniques to address missing values, including imputation, which involves substituting statistical estimates such as the mean or median for the missing values and deleting rows or columns containing missing data. To preserve the integrity of the dataset and avoid biases or mistakes during model training, this step is essential.

Feature Encoding: A lot of machine learning techniques need numerical data. However categorical data in datasets frequently has to be transformed into a numerical representation. Categorical variables were converted into binary vectors using OneHotEncoder, where each category is represented by a distinct column. To make sure that the machine learning algorithms can appropriately process categorical information, this procedure—known as one-hot encoding—is crucial.

Discretization: Data_Value, the goal variable in the dataset, is a representation of mental health outcomes. The continuous Data_Value was discretized into discrete categories to reduce the complexity of the prediction challenge to a classification problem. This was accomplished by using KBinsDiscretizer, which converted a continuous output into a categorical one by splitting the data range into bins and allocating each data point to a bin. This strategy works especially well in situations when categorizing results instead of treating them as continuous numbers allow for better meaningful interpretation of the results.

MODEL EVALUATION METRICS

Several assessment metrics were employed to evaluate these models' performance:

Accuracy: Accuracy is the percentage of accurately predicted cases out of total cases. Accuracy by itself can be deceptive, even though it gives a rough idea of the model's performance particularly when there is a class imbalance.

Precision: Out of all positive forecasts, precision represents the percentage of true positive predictions. Because it concentrates on the accuracy of positive classifications, it is essential in situations where the cost of false positives is significant.

Recall: Recall, also called sensitivity, quantifies the percentage of true positives that the model correctly identified. It is significant in situations where omitting a positive case (false negative) is crucial, like in medical diagnosis.

F1 Score: The F1 score is the harmonic mean of precision and recall, offering a fair assessment that takes into account both false positives and false negatives. It is especially helpful in situations where there are imbalanced classes, as it provides a more nuanced picture of the model's performance than accuracy alone.

3.2 RESEARCH DESIGN

This study's research approach uses the "Alzheimer's Disease and Healthy Ageing Data" dataset to systematically assess how well machine learning models predict mental health measures across older populations. Data collection and preprocessing are the first steps, which include data cleaning, column removal, management of missing values, combining data related to years, and categorical variable encoding. For analysis, important attributes like Year, Stratification1, Stratification2, Class, and LocationDesc are chosen. Multiple machine learning models are used in the study, with the target variable Data_Value being discretized into categories. These models include Logistic Regression, Random Forest Classifier, Gradient Boosting Classifier, Support Vector Classifier, Decision Tree Classifier, and KNN. Metrics like accuracy, precision, recall, and F1-score are used to assess the models' performance once they have been trained on the dataset. The best model for forecasting measures of mental health is found through comparison analysis. Next, to confirm the trained models' practical applicability, they are tested on fresh, untested data samples. The findings highlight how machine learning may be used to provide tailored therapies and optimize the use of resources in mental health care, offering insightful

information to politicians and healthcare practitioners. To further enhance forecast accuracy and applicability, the paper makes recommendations for future research paths that center on model improvement and the investigation of cutting-edge methods.

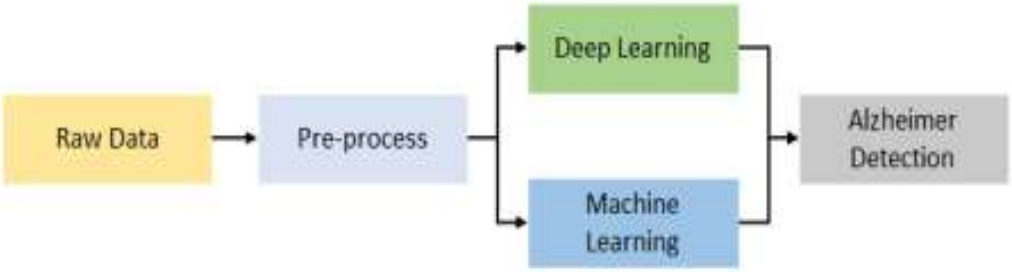


Figure 9: Alzheimer's Disease Workflow

3.3 DATA SOURCE

The "Alzheimer's Disease and Healthy Ageing Data" dataset (https://catalog.data.gov/dataset/alzheimers-disease-and-healthy-aging-data) is a comprehensive source of health-related metrics emphasizing aging populations, which is used for data collecting in this study. The dataset is regarded as secondary data. This indicates that other researchers or institutions were the ones who initially collected information. There are both quantitative and qualitative data in this dataset i,e geographic data that describes specific locations, demographic data like age and gender, and a variety of mental health variables that are essential for evaluating cognitive decline and frequent mental anguish in the elderly. It also includes temporal data, which charts changes in health across time. So basically the names of the columns in the dataset are like "Year", "Stratification1", "Stratification2", "Class", and "LocationDesc". The information is gathered using standardized survey questionnaires, clinical evaluations, and self-reported measurements, guaranteeing wide and trustworthy representation. It is derived from public health surveys and research projects carried out by health authorities.

3.4 TOOLS AND TECHNOLOGIES

To process data, build and evaluate machine learning models, and visualize findings, a variety of tools and technologies are used in this work. With tools like Pandas and NumPy for numerical computing and data manipulation, Python is the most often used programming language. To handle categorical data and discretize continuous

variables, respectively, OneHotEncoder and KBinsDiscretizer are utilized for data preprocessing; ColumnTransformer assists in applying diverse preprocessing strategies to different data features. Utilizing the sci-kit-learn library, the machine learning models created include Support Vector, Gradient Boosting, Random Forest, Logistic Regression, Decision Tree Classifier, and KNN.

Accuracy, precision, recall, and F1-score are some of the performance measures that are used to evaluate these models. The Classification Report and Confusion Matrix are two tools that are useful for doing precise performance analysis. To make plots and graphs that are instructive, visualization packages like Matplotlib and Seaborn are used. Package management and deployment are made easier by the use of the Anaconda Distribution's Spyder interactive computing environment for conducting research. To conduct predictive modeling in the study of mental health indicators among aging populations, robust analysis and efficient processing of data are made possible by this extensive array of tools and technologies.

3.5 ETHICAL CONSIDERATION

Multiple ethical considerations are crucial when using the "Alzheimer's Disease and Healthy Ageing Data" dataset to forecast mental health indicators in older people. Ensuring data privacy and confidentiality is essential, requiring tight compliance with data protection laws and data anonymization to safeguard individual identities. The dataset should only be utilized if it was gathered with participants' informed agreement after they were fully informed of the goals of the study and any potential repercussions. To prevent misinterpretation or abuse and to guarantee that results are analyzed by licensed healthcare professionals, the implementation of machine learning models especially in delicate areas like mental health must be done properly. To avoid increasing already-existing differences, regular audits for fairness and equity are crucial in mitigating bias in the data and models. For reproducibility and accountability, technique transparency is required, including data cleaning, model selection, and evaluation procedures. While avoiding stigmatization or discrimination and making sure the results are reported ethically and clearly, researchers must also take into account the influence their findings will have on society. To guarantee that the information is used appropriately and that participants' rights are upheld, data sharing must adhere to ethical and legal requirements. To

ensure that research meets high standards and makes a beneficial contribution to the field of mental health, ethical considerations are crucial to the process.

3.6 RESEARCH LIMITATIONS

There are several restrictions on the research, most of which have to do with generalisability and data quality. The accuracy and dependability of the conclusions may be impacted by missing data, reporting biases, or inaccuracies in the "Alzheimer's Disease and Healthy Ageing Data" dataset. The conclusions' ability to be broadly applied may also be limited by the dataset's potentially incomplete representation of the larger population. There are inherent assumptions and limitations with the machine learning models that are used, like Support Vector Classifier, Gradient Boosting Classifier, Random Forest Classifier, and Logistic Regression. These include potential problems with overfitting or underfitting as well as difficulties interpreting intricate ensemble methods. Another issue with the data is its temporal validity, which could affect how relevant the results are in the future since it represents a particular period.

The use of sensitive mental health data raises ethical and privacy issues as well because even with anonymization, there remains a chance of re-identification. The study's depth and breadth may be limited by a lack of resources, and the emphasis on predicting accuracy alone may obscure other crucial aspects like usability, affordability, and practicality. The dataset's emphasis on mental health and aging populations may also fail to take into consideration other significant characteristics like lifestyle, physical health, or socioeconomic status, which could restrict how thorough the study's findings can be. Accurately interpreting the data and guiding future research approaches depend on an understanding of these constraints.

4 RESULTS AND IMPLEMENTATION

4.1 Data Preprocessing

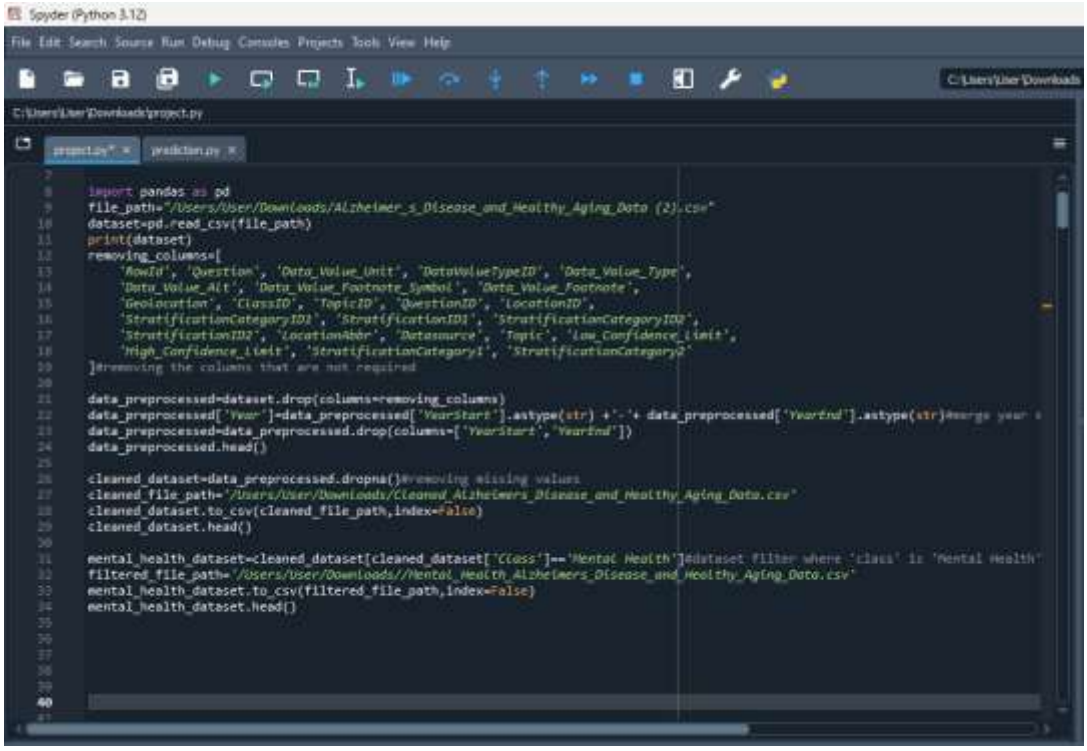


Figure 10:Data Preprocessing

The code above preprocesses a dataset about healthy aging and Alzheimer's disease. The Pandas library is first used to load the dataset from a CSV file. To make the data easier to analyze, superfluous columns are found and eliminated. After that, the code combines "YearStart" and "YearEnd" into a single "Year" column to make the temporal analysis simpler. The dataset is cleaned and saved to a new CSV file after rows containing missing values are removed to guarantee data integrity. The information is then further refined to include only those rows in which the 'Class' column indicates 'Mental Health,' therefore concentrating the dataset on pertinent mental health indicators. Additionally, for additional analysis, this subset is saved to a different CSV file. To ensure that the analysis is effective and focused, preprocessing procedures such as filtering, data cleaning, and column removal are essential for preparing the data for machine learning and statistical modeling tasks that follow.

```
Console 1/A
284140    BRFSS~2019~2019~54~Q46~TOC10~AGE~RACE  ...                HIS
284141    BRFSS~2015~2015~56~Q02~TNC02~AGE~RACE  ...                NAA

[284142 rows x 31 columns]
Out[5]:
                        LocationDesc  ...       Year
3                           Maryland  ...  2022-2022
4                          Wisconsin  ...  2022-2022
6                           Oklahoma  ...  2022-2022
7                       Pennsylvania  ...  2022-2022
14  United States, DC & Territories   ...  2022-2022

[5 rows x 6 columns]

In [6]:
IPython Console   History
```

Figure 11:Data Summary

This output, which is a sample from a pandas Data Frame, contents of the cleaned dataset following certain stages of data transformation and cleaning. The dataset's original dimensions, 284,142 rows, and 31 columns, are indicated by the notation[284142 rows * 31 columns]. The cleaned dataset includes five rows and six columns, indicated by the notation[5 rows * 6 columns].

4.2 Random Forest Classifier

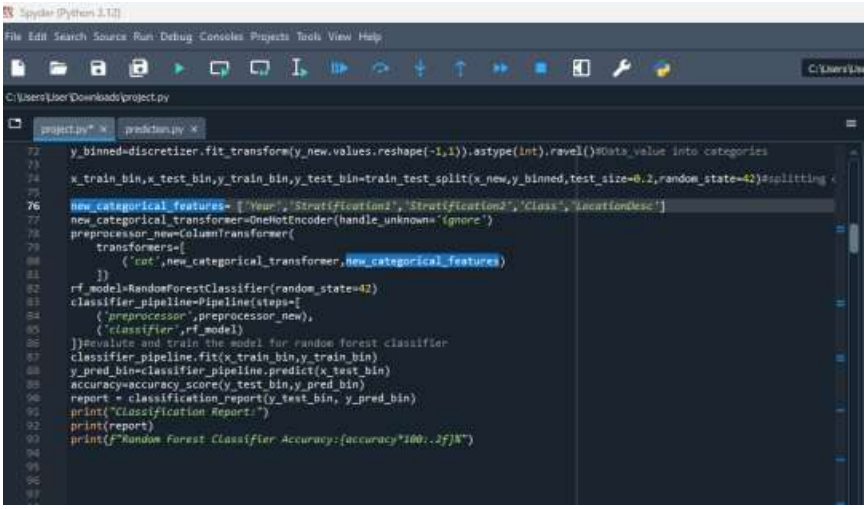
```
Spyder (Python 3.12)
File Edit Search Source Run Debug Consoles Projects Tools View Help
C:\Users\User\Downloads\project.py
project.py*   prediction.py

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, KBinsDiscretizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

new_features=['Year','Stratification1','Stratification2','Class','LocationDesc']
target='Data_Value'

x_new=mental_health_dataset[new_features]
y_new=mental_health_dataset[target]

n_bins=5
discretizer=KBinsDiscretizer(n_bins=n_bins,encode='ordinal',strategy='uniform')
y_binned=discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()#Data_value into categories

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=0.2,random_state=42)#splitting
```

Figure 12:Random Forest Classifier

This small block of code belongs to a machine-learning process that uses a Random Forest Classifier to predict mental health variables. The first step is to load the

necessary libraries, which include Seaborn and Matplotlib for visualization, pandas for data manipulation, and scikit-learn for data preprocessing, training, and assessment. "Year," "Stratification1," "Stratification2," "Class," and "LocationDesc" are the features that are to be employed in the model; the goal variable is "Data_Value." The dataset is divided into the target (y_new) and features (x_new), collectively known as the mental_health_dataset. For classification tasks, the KBinsDiscretizer from scikit-learn is used to convert the continuous target variable into a categorical representation. To apply the Random Forest Classifier, this discretizer transforms Data_Value into five ordinal categories.

Discretization is especially crucial in situations where the modeling approach calls for categorical results while the target variable is continuous. Next, to ensure that the model's performance is assessed on unobserved data, the train_test_split function splits the data into training and testing sets, allocating 20% of the data for testing and the remaining portion for training. Reliable results and the ability for others to duplicate the study depend on the splitting process being consistent and repeatable, which is ensured by the parameter random_state=42. These processes aim to prepare the dataset in a way that optimizes the predicted accuracy of the model and facilitates a thorough assessment of its performance. To minimize the possibility of errors or inaccurate predictions in the model, this preparation stage of any machine learning process is essential since it establishes the groundwork for model training and guarantees that the data is properly organized and clean. Not covered in this excerpt are the processes that would come after preprocessing and modeling are set up, the Random Forest model is trained, and its accuracy, precision, recall, and F1-score are assessed to obtain a thorough grasp of its predictive power.

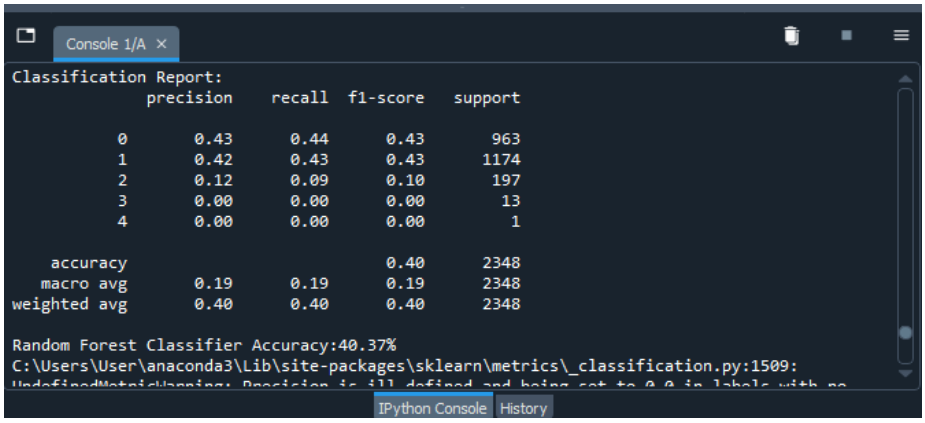


```
y_binned=discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()#Data value into categories

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=0.2,random_state=42)#splitting

new_categorical_features= ['Year','Stratification1','Stratification2','Class','LocationDesc']
new_categorical_transformer=OneHotEncoder(handle_unknown='ignore')
preprocessor_new=ColumnTransformer(
    transformers=[
        ('cat',new_categorical_transformer,new_categorical_features)
    ])
rf_model=RandomForestClassifier(random_state=42)
classifier_pipeline=Pipeline(steps=[
    ('preprocessor',preprocessor_new),
    ('classifier',rf_model)
])#evalute and train the model for random forest classifier
classifier_pipeline.fit(x_train_bin,y_train_bin)
y_pred_bin=classifier_pipeline.predict(x_test_bin)
accuracy=accuracy_score(y_test_bin,y_pred_bin)
report = classification_report(y_test_bin, y_pred_bin)
print("Classification Report:")
print(report)
print(f"Random Forest Classifier Accuracy:{accuracy*100:.2f}%")
```

Figure 13:Encoding Categorical Features

This Python code describes a thorough procedure for configuring, training, and assessing a Random Forest Classifier through a machine-learning pipeline. ‘Year’, ‘Stratification1’, ‘Stratification2’, ‘Class’, and ‘LocationDesc’ are the categorical features selected from the dataset. This is the first step in the procedure. OneHotEncoder, which transforms categorical data into a binary matrix format appropriate for machine learning models, is used to change these characteristics. By ensuring that the encoder safely ignores any unknown categories discovered during the transformation, the handle_unknown='ignore' argument helps to prevent potential problems during testing. Following the transformation of the features, a ColumnTransformer is incorporated into the process, standardizing the preprocessing phase and guaranteeing uniform data transformation in the training and prediction stages. To guarantee the reproducibility of results, a RandomForestClassifier is instantiated with a specified random_state after the preprocessing setup. This classifier is a component of an ensemble learning technique that increases prediction robustness and accuracy by utilizing numerous decision trees. A Pipeline, a useful tool that automates the series of transformations and model training, is created by combining the preprocessing stages and the classifier. Since it guarantees consistency and integrity by applying the same preprocessing to the training and test datasets, this pipeline is especially helpful. The final product provides information on the model’s overall accuracy as well as a comprehensive classification report that highlights the model’s advantages and shortcomings. This procedure highlights the significance of consistent data

preparation, strong model training, and comprehensive evaluation to guarantee dependable and understandable outcomes. It is an example of an organized and effective way to develop a predictive model.



```
Classification Report:
              precision    recall  f1-score   support

           0       0.43      0.44      0.43       963
           1       0.42      0.43      0.43      1174
           2       0.12      0.09      0.10       197
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1

    accuracy                           0.40      2348
   macro avg       0.19      0.19      0.19      2348
weighted avg       0.40      0.40      0.40      2348

Random Forest Classifier Accuracy:40.37%
C:\Users\User\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1509:
```

Figure 14: Classification Report for RFC

The output offers a comprehensive evaluation of the effectiveness of a Random Forest Classifier, emphasizing important metrics like recall, precision, and F1-score for every class. With an accuracy of 40.37% overall, the model was able to predict the target class for 40.37% of the test samples. While classes with fewer instances (2, 3, and 4) exhibit much lower accuracy and recall, especially class 3 and 4, which have an F1-score of 0, the most common classes (0 and 1) have precision and recall around 0.43, indicating average competence. While the weighted average accounts for the number of occurrences in each class, providing a more balanced assessment, the macro average delivers an overall mean score, considering all classes equally.

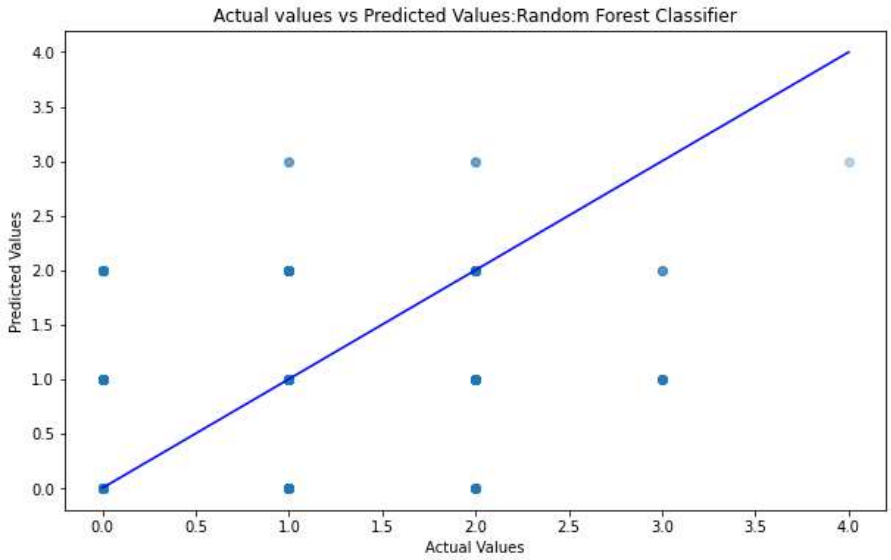


Figure 15:Actual vs Prediction plot for RFC

The scatter plot, which shows the real values on the x-axis and the predicted values on the y-axis, shows the relationship between the actual values and the predicted values from a Random Forest Classifier. When expected and actual values coincide exactly, perfect accuracy is represented by the diagonal blue line. Ones that are near this line represent the model's correct predictions, while ones that are far from it represent situations in which the model's forecasts greatly deviate from the actual results. While some predictions are correct, the distribution of the data points around the line indicates that the model's performance is not constant.

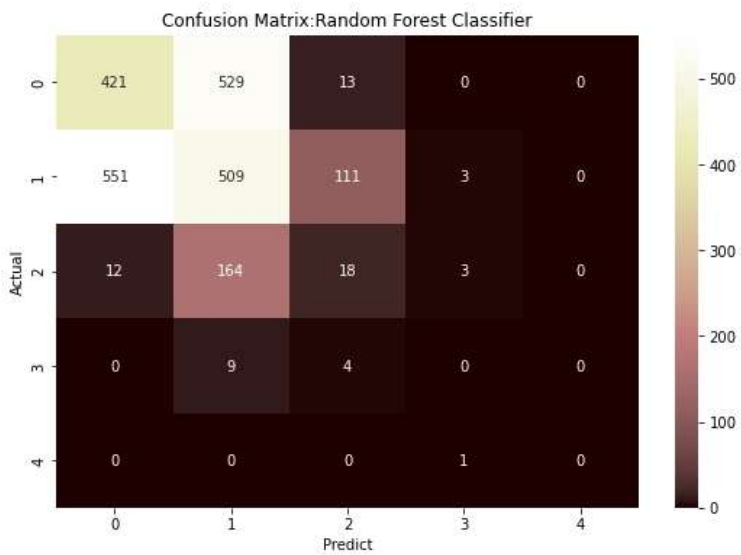
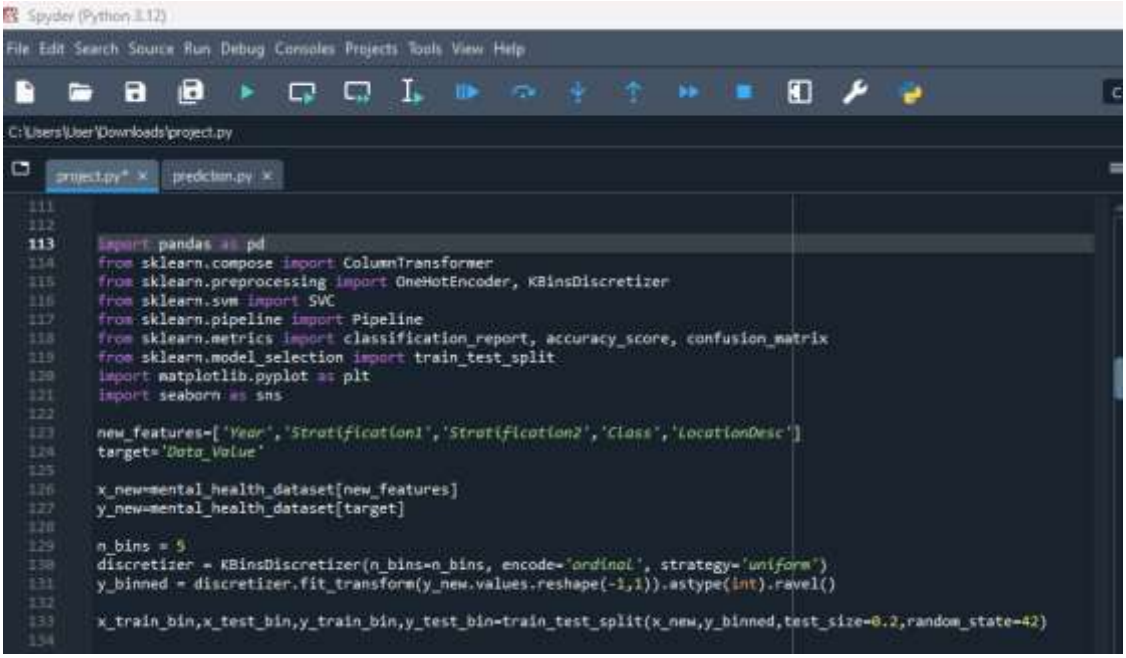


Figure 16:Confusion Matrix for RFC

The confusion matrix shows how well a Random Forest Classifier performs, with rows denoting actual classes and columns predicted classes. Class 0 correctly predicted with 421 and class 1 predicted with 509, respectively, whereas class 0 misclassified with 529 instances of class 0 misclassified as class 1 is indicated by off-diagonal cells. As seen by the large number of misclassifications, especially between classes 0 and 1, the model performs well for some classes but fails miserably with others.

4.3 SUPPORT VECTOR CLASSIFIER



```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, KBinsDiscretizer
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns

new_features=['Year','Stratification1','Stratification2','Class','LocationDesc']
target='Data_Value'

x_new=mental_health_dataset[new_features]
y_new=mental_health_dataset[target]

n_bins = 5
discretizer = KBinsDiscretizer(n_bins=n_bins, encode='ordinal', strategy='uniform')
y_binned = discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=0.2,random_state=42)
```

Figure 17::Support Vector Classifier Model Setup

This snippet of code configures a Support Vector Classifier (SVC)-based machine learning pipeline to predict mental health variables. Importing the required libraries for data preprocessing, modeling training, and evaluation is the first step. The dataset is used to pick the features “Year”, “Stratification1”, “Stratification2”, “Class”, and “LocationDesc”, with Data_Value serving as the target variable. This target variable is converted from continuous to categorical using KBinsDiscretizer, which bins the data into five groups for classification purposes. After that, 20% of the data is put aside for testing, and random_state=42 is used to ensure repeatability. The data is then divided into training and testing sets using a train test split. Setting up

the basis for model training and testing, including the creation of metrics like accuracy, confusion matrix, and classification report, requires this preprocessing and data separation to train the SVC model and evaluate its performance on unseen data.

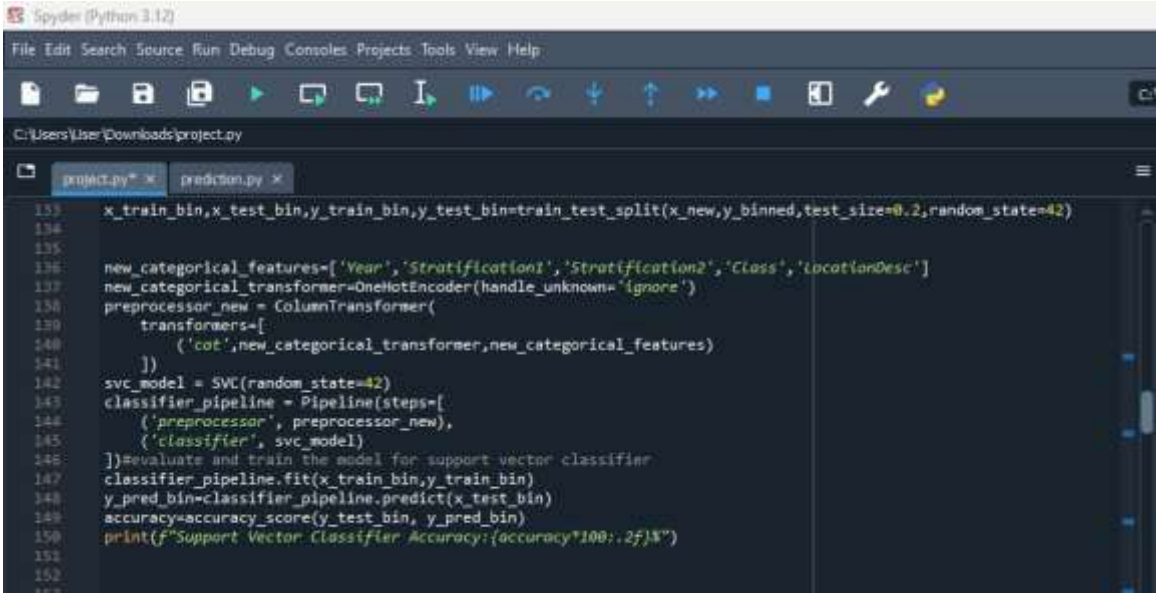


Figure 18: Encoding Categorical for Support Vector Classifier

This piece of code simplifies the process of training a model and preprocessing data by setting up and optimizing a Support Vector Classifier (SVC) using a pipeline. It begins by defining the categorical features (Year, Stratification1, Stratification2, Class, and LocationDesc) and then uses OneHotEncoder to transform them into a numerical format that the classifier can understand. This encoding is a component of the ColumnTransformer, which guarantees uniform preprocessing throughout the dataset. Next, the two primary parts of the pipeline are built: the SVC model for classification and the ColumnTransformer for preprocessing. Data consistency and integrity are ensured by applying the same data transformations via the pipeline during the training and testing stages. By using accuracy_score to assess the prediction accuracy, the accuracy % that is obtained is reported. By demonstrating the model's ability to accurately forecast the target variable using the given information, this arrangement offers a methodical strategy for developing and assessing the SVC model.

```
Console 1/A ×
Classification Report:
              precision    recall  f1-score   support

           0       0.62      0.63      0.63       963
           1       0.59      0.67      0.63      1174
           2       0.42      0.04      0.07       197
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1

    accuracy                           0.60      2348
   macro avg       0.33      0.27      0.27      2348
weighted avg       0.58      0.60      0.58      2348

Support Vector Classifier Accuracy:60.05%
```

Figure 19:Classification Report of SVM

A support vector classifier's performance is evaluated in the output, which displays an overall accuracy of 60.05%, or that the model correctly identified the target class in 60.05% of the test samples. For every class, the model's precision, recall, and F1 score are broken down to show how well it can predict each class. For example, for classes 0 and 1, the model performs moderately, with precision and recall of approximately 0.62 and 0.67, respectively. Class 2, 3, and 4 are the worst performs; in particular, class 3 and class 4 have precision and recall of zero, meaning the model was unable to accurately identify these classes. The model struggles with minority classes, as evidenced by lower scores in the macro average, which treats all classes equally. Weighted averages indicate the model's tendency towards more prevalent classes, but they also offer a more balanced view by accounting for the number of examples for each class.

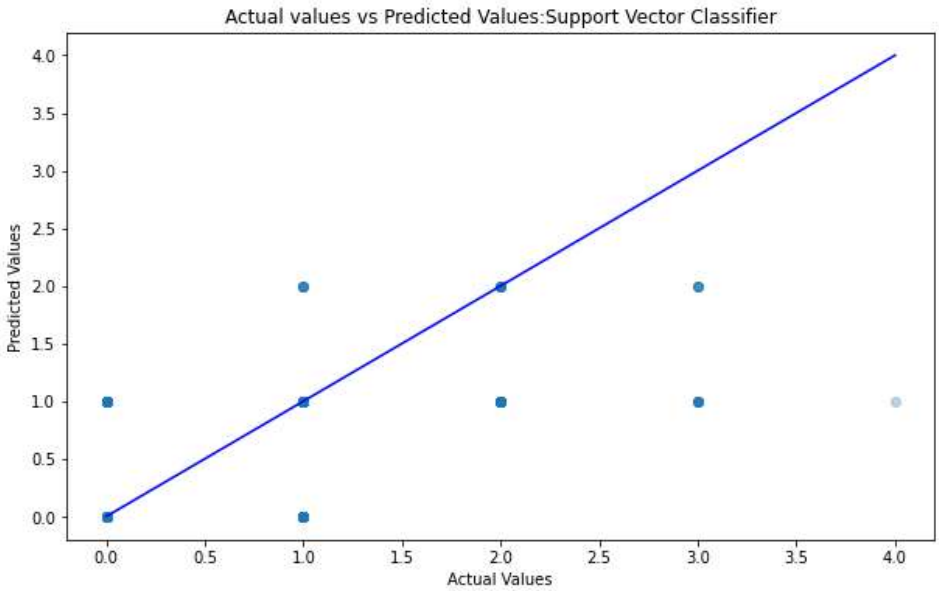


Figure 20: Actual vs Predicted plot for SVM

The scatter plot shows the correlation between the values that a support vector classifier predicts and the actual values. The y-axis shows the expected values of the target variable, and the x-axis shows the actual values. The ideal prediction range, or the point at which the expected and actual values coincide exactly, is shown by the blue diagonal line. While data points distant from the line suggest mispredictions, data points near this line illustrate situations in which the model's predictions are correct. The distribution of points away from the line indicates that there is variation in the model's performance, with some predictions being notably correct and others not.

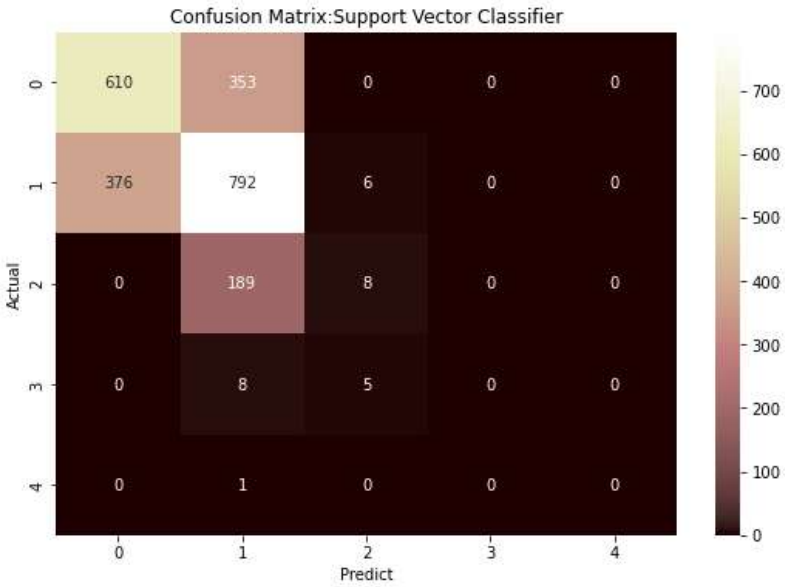


Figure 21:Confusion Matrix Of SVM

The Support Vector Classifier's confusion matrix plots actual classes on the y-axis and predicted classes on the x-axis, and displays the model's performance across various classes. Correct predictions are shown by the diagonal cells, where 610 cases are correctly classified as class 0, 792 as class 1, 8 as class 2, and 5 as class 3. Class 0 and class 1 were confused, as seen by the 353 class 0 occurrences that were incorrectly categorized as class 1 and the 376 class 1 examples that were incorrectly classed as class 0 in off-diagonal cells. Moreover, 189 cases of class 2 were mistakenly assigned to class 1. Class 3 and class 4 have sparse numbers, and class 4 has zero valid predictions, indicating that the model has a lot of difficulty with these less common classes.

4.4 GRADIENT BOOSTING CLASSIFIER

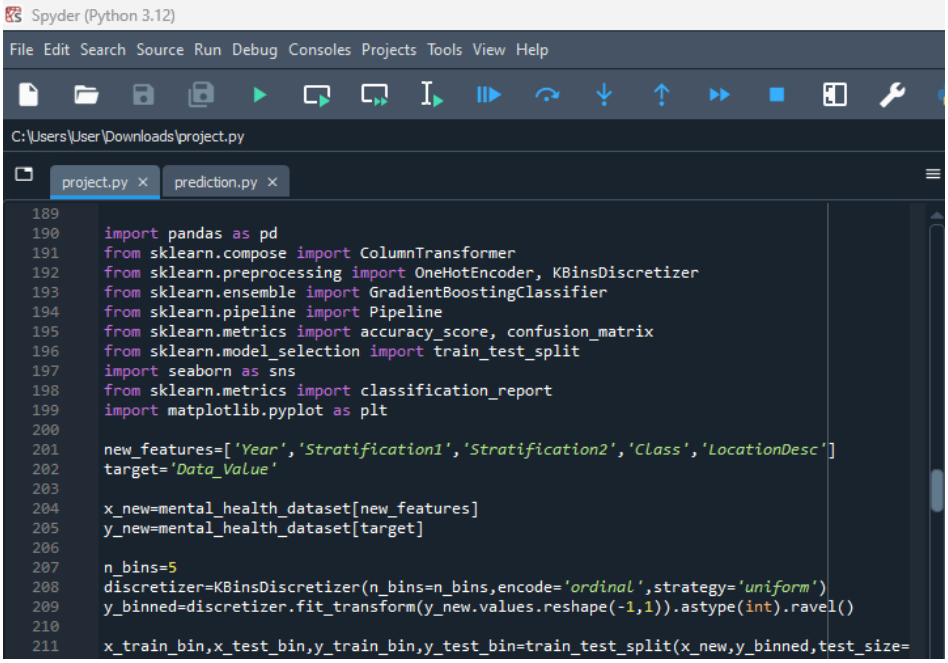


Figure 22: Gradient Boosting Classifier Setup

This snippet of code is a component of a machine learning pipeline that uses Python's scikit-learn package to analyze and forecast mental health variables using a Gradient Boosting Classifier. Starting with the mental_health_dataset, the workflow selects the appropriate characteristics “Year”, “Stratification1”, “Stratification2”, “Class”, and “LocationDesc” using 'Data_Value' as the target variable. With the KBinsDiscretizer, the data is divided into five bins of identical width, converting the continuous target variable into discrete categories appropriate for classification. This transformation turns the target into numerical categories, making it easier to apply a classification model. After that, the train test split divides the dataset into training and testing sets, allocating 20% of the data for testing to assess the performance of the model. A crucial element in obtaining consistent findings is the reproducibility of the data split, which is ensured by the setting random_state=42. Crucially, this preparatory stage lays the groundwork for training the Gradient Boosting Classifier and the subsequent model evaluation, which measures the classifier's efficacy using metrics like accuracy, precision, recall, and F1-score.

```
x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=

new_categorical_features=['Year','Stratification1','Stratification2','Class','LocationDe
new_categorical_transformer=OneHotEncoder(handle_unknown='ignore')
preprocessor_new=ColumnTransformer(
    transformers=[
        ('cat',new_categorical_transformer, new_categorical_features)
    ])#evaluate and tain the model for Gradient Boosting Classifier
classifier_pipeline=Pipeline(steps=[
    ('preprocessor',preprocessor_new),
    ('classifier',GradientBoostingClassifier(random_state=42))
])
classifier_pipeline.fit(x_train_bin, y_train_bin)
y_pred_bin=classifier_pipeline.predict(x_test_bin)
accuracy = accuracy_score(y_test_bin, y_pred_bin)
report = classification_report(y_test_bin, y_pred_bin)
print("Classification Report:")
print(report)
print(f"Gradient Boosting Classifier Accuracy:{accuracy*100:.2f}%")
```

Figure 23:Encoding Categorical for GBM

This snippet of code shows how to use a pipeline to set up, train, and evaluate a Gradient Boosting Classifier in Python with an emphasis on categorical data preprocessing. The process starts with the identification of important categorical features including "Year," "Stratification1," "Stratification2," "Class," and "LocationDesc." OneHotEncoder is then used to convert these features into a binary matrix format. To prevent mistakes during prediction, the handle_unknown='ignore' argument makes sure that any categories that were not encountered during training. This transformation is applied uniformly across these characteristics using a ColumnTransformer. Preparing the data in a numerical format appropriate for the machine learning model requires this preprocessing step.

```
  warnings.warn(
Classification Report:
              precision    recall  f1-score   support

           0       0.65      0.60      0.62       963
           1       0.60      0.71      0.65      1174
           2       0.37      0.10      0.15       197
           3       0.17      0.08      0.11        13
           4       0.00      0.00      0.00         1

    accuracy                           0.61      2348
   macro avg       0.36      0.30      0.31      2348
weighted avg       0.60      0.61      0.59      2348

Gradient Boosting Classifier Accuracy:60.95%
```

Figure 24:Classification Report of GBM

The Gradient Boosting Classifier's performance is assessed in the classification report, which shows an overall accuracy of 60.95%, or that 60.95% of the test samples were correctly predicted by the model. For every class, the report provides a breakdown of performance indicators including recall, precision, and F1-score. F1-score strikes a balance between recall, which shows how effectively the model recognises real positive cases, and precision, which quantifies the accuracy of the positive predictions. For classes 0 and 1, the model performs moderately, with recall ranging from 0.60 to 0.71 and precision around 0.65. However, it fails miserably in classes 2, 3, and 4, especially in class 4, where it fails to forecast this class at all, as evidenced by its 0 precision, recall, and F1 score. The weighted average takes into consideration the number of instances per class, whereas the macro average treats all classes equally. These two averages demonstrate how difficult it is for the model to handle less represented classes. According to this, the model suffers greatly from minority classes even though it can predict the more common classes with reasonable accuracy. This highlights development opportunities, such as increasing the model's sensitivity to these less common categories.

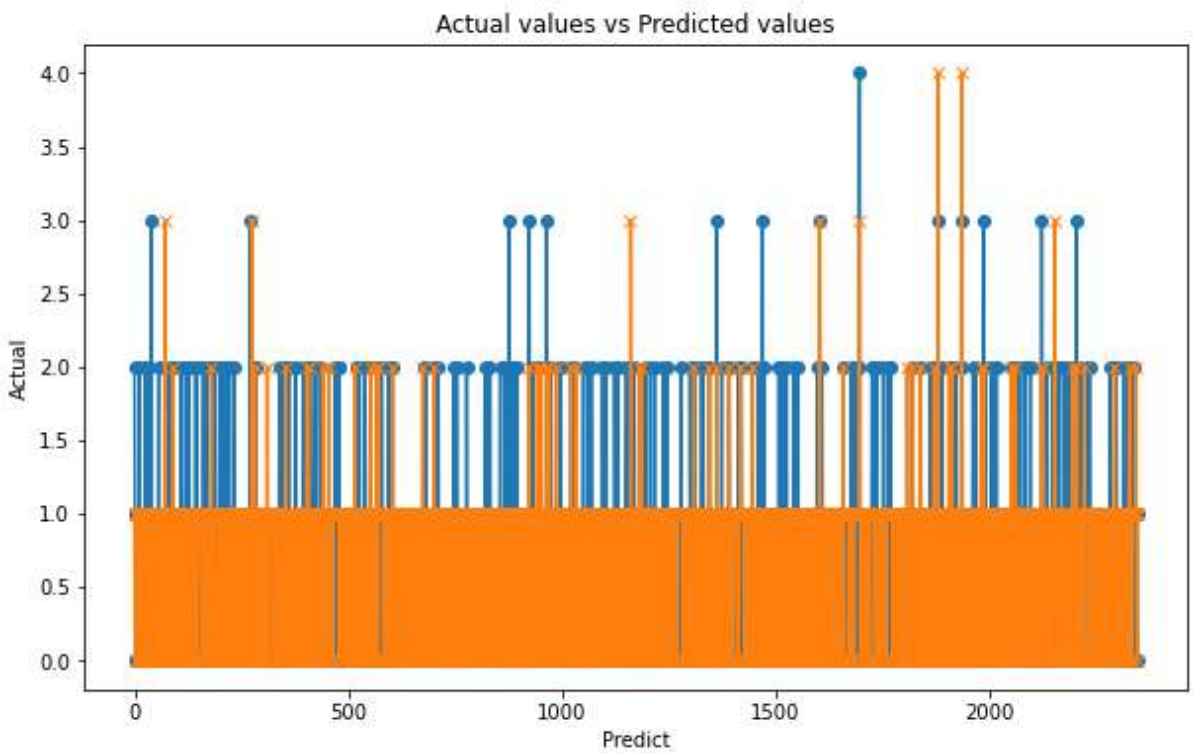


Figure 25:Actual vs Prediction plot of GBM

Depending on prior information, a machine learning model—likely a Gradient

Boosting Classifier—predicts values depending on what is shown in the graph. Labelled "Predict," the x-axis most likely represents the predicted values or sample index, and labelled "Actual," the y-axis displays the actual target values. The difference between each vertical line's top and bottom represents the actual value and the anticipated value, respectively. It is possible to distinguish between accurate and inaccurate forecasts if both blue and orange colours are present, as they could signify distinct data points or classes. The graph indicates the ability of the model to understate some values by displaying predictions towards the lower end of the actual value scale. The vertical spread of the lines depicts the model's prediction accuracy; bigger spreads signify larger differences between actual and anticipated values, which correspond to less accurate forecasts.

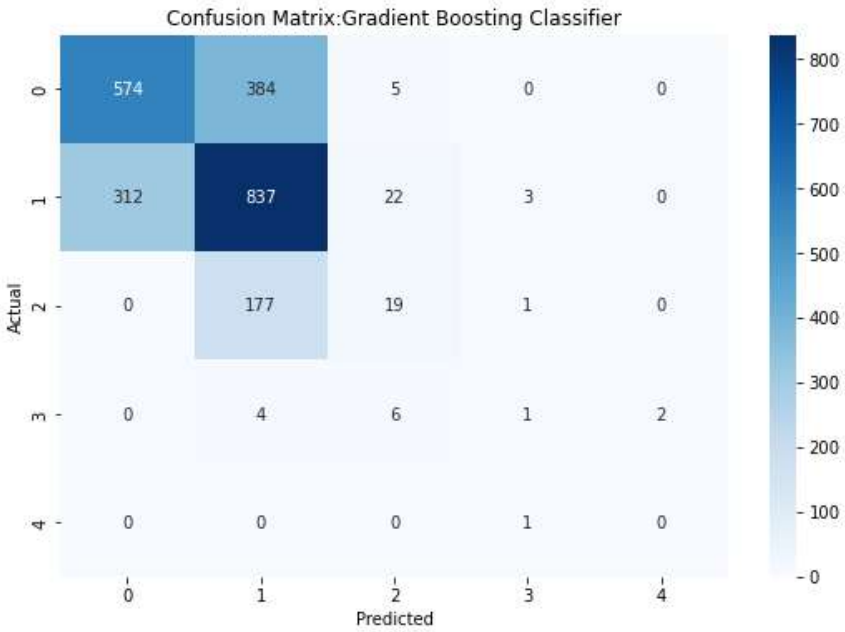
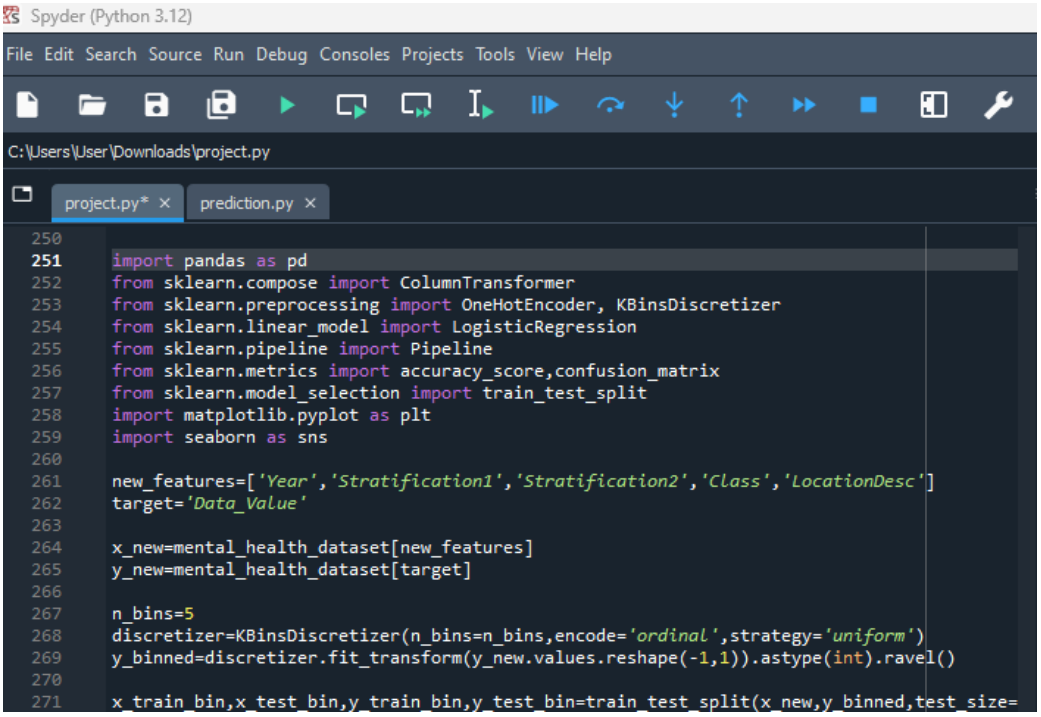


Figure 26:Confusion Matrix of GBM

The Gradient Boosting Classifier's confusion matrix compares actual against predicted labels to show how well the model performs across different classes. With 574 cases correctly identified as class 0 and 837 as class 1, the diagonal values represent valid predictions, showing a respectable level of accuracy for these classes. But there are clear significant misclassifications: 384 cases of class 0 were predicted to be class 1, while 312 cases of class 1 were expected to be class 0. The matrix also shows how hard it is for the model to forecast less common classes,

especially class 2, which is frequently mistakenly assigned to class 1, and classes 3 and 4, which have very few accurate predictions. These patterns are highlighted by the colour intensity, where higher counts are shown by darker hues. Overall, the matrix indicates areas that could use improvement: although the classifier does rather well for the more common classes, it has trouble differentiating between similar classes and correctly predicting the less common ones.

4.5 LOGISTIC REGRESSION




```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, KBinsDiscretizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score,confusion_matrix
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns

new_features=['Year','Stratification1','Stratification2','Class','LocationDesc']
target='Data_Value'

x_new=mental_health_dataset[new_features]
y_new=mental_health_dataset[target]

n_bins=5
discretizer=KBinsDiscretizer(n_bins=n_bins,encode='ordinal',strategy='uniform')
y_binned=discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=
```

Figure 27:Logistic Regression Model Setup

This snippet of code prepares data for the training of a machine learning model, which is likely to be logistic regression, to predict mental health measures. It begins by loading the scikit-learn and pandas packages that are required for data manipulation and model construction. Using "Data_Value" as the target variable, the mental health dataset is processed to extract pertinent properties including "Year," "Stratification1," "Stratification2," "Class," and "LocationDesc." KBinsDiscretizer is used to convert the target variable, which was previously continuous, into categorical data by binning the data into five ordinal categories. To make the regression problem more appropriate for models such as logistic regression, this transformation aids in transforming it into a classification task. To enable the evaluation of the model on

data that has not yet been seen, the data is then divided into training and testing sets, with 80% designated for training and 20% for testing. For consistent findings over multiple runs, the random_state=42 parameter guarantees the repeatability of the data split. Effective model training and evaluation depend on the proper data structure, which is achieved through this preparatory step.

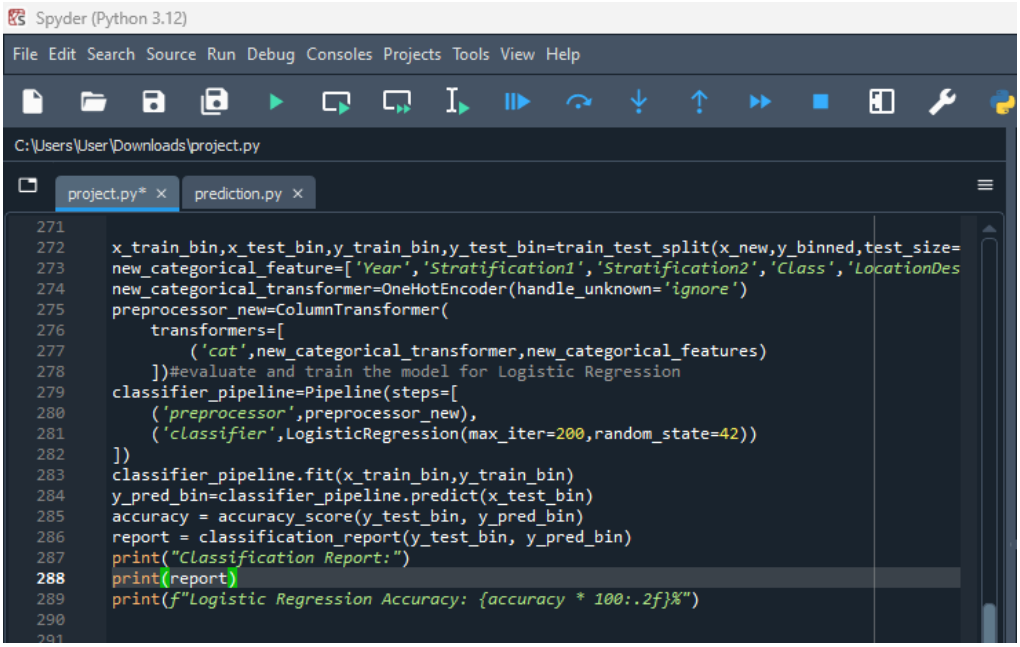


Figure 28:Encoding Categorical for Logistic Regression

A pipeline that combines data before treatment and model training is used in this code snippet to show how a Logistic Regression model can be trained and evaluated. Class, LocationDesc, Stratification1, Stratification2, Year, and other category characteristics are defined first. OneHotEncoder is then used to convert these categories into a binary format that can be used for modelling. A uniform preparation of the data is ensured by applying the transformation via a ColumnTransformer. Using a fixed random state for repeatability and a maximum of 200 iterations for convergence on the Logistic Regression model, a Pipeline is built to optimise the procedure. The training data (x_train_bin, y_train_bin) are used to train the model, which is subsequently used to predict the test data (x_test_bin). Classification_report, which offers specific metrics like precision, recall, and F1-score for each class, and accuracy_score, which calculates the proportion of accurate predictions, are used to assess the model's performance.

```
Console 1/A ×
Classification Report:
              precision    recall  f1-score   support

           0       0.65      0.60      0.63       963
           1       0.60      0.71      0.65      1174
           2       0.43      0.18      0.26       197
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1

    accuracy                           0.62      2348
   macro avg       0.34      0.30      0.31      2348
weighted avg       0.61      0.62      0.60      2348

Logistic Regression Accuracy: 61.63%
```

Figure 29:Classification Report of Logistic Regression

The classification report analyses the effectiveness of a logistic regression model, demonstrating an overall accuracy of 61.63%, or 61.63% of test samples with the target class correctly predicted by the model. The report provides insights into how well the model distinguishes between various categories by breaking down precision, recall, and F1 scores for each class. For example, for classes 0 and 1, the model performs rather well, with recall ranging from 0.60 to 0.71 and precision around 0.65. However, it suffers in class 2, where precision and recall are lower, and does not do well in classes 3 and 4, where the model produces zero precision and recall, indicating that no predictions are made correctly. Though the weighted average takes into account the class distribution and shows that the model's better accuracy on the more frequent classes influences its overall performance, the macro average, which treats all classes equally, shows lower values, reflecting the model's poor performance on minority classes. The graph shows the difference between the predicted values (represented by the y-axis) and the actual values (represented by the x-axis) of a logistic regression model. The difference between the pridicted value and the actual value for each vertical line represents the prediction error of the model. Although it's not immediately apparent how exactly they are classified, the usage of orange and blue colours may help distinguish between accurate and inaccurate forecasts. Many vertical lines, especially ones that move widely from the baseline, indicate considerable differences between the actual and anticipated values, hence emphasising prediction mistakes. This is particularly noticeable in some classes or ranges where the larger vertical dispersion suggests that the model

may be less accurate.

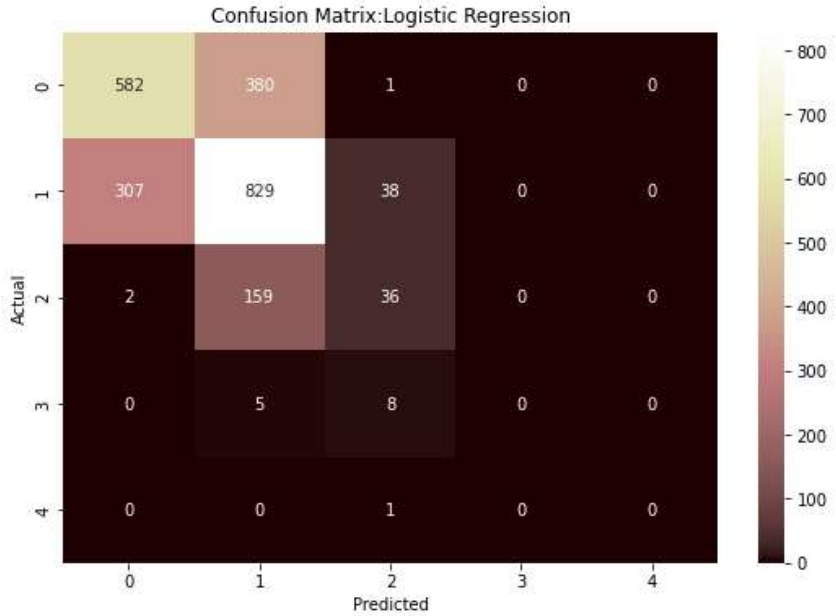


Figure 30:Confusion Matrix of Logistic Regression

The Logistic Regression model's confusion matrix offers a comprehensive overview of the model's ability to categorise data into different categories. The y-axis of the matrix shows real class labels, while the x-axis shows expected class labels. Where the model works well is indicated by the diagonal elements, which show the number of instances properly classified by the model for each class. For instance, 829 cases of class 1 and 582 instances of class 0 were accurately identified by the model. Misclassifications are represented as off-diagonal elements, which highlight instances in which the model's predictions deviate from the real classes. Notably, there are 307 occurrences of class 1 predicted as class 0 and 380 cases of class 0 mistakenly forecasted as class 1, demonstrating misunderstanding between these two classes. Additionally, the matrix reveals a considerable inaccuracy in class 2, with 159 examples incorrectly predicted as class 1 and 36 as class 2, suggesting that the model has difficulty making these distinctions. Darker hues in the matrix indicate higher counts, and their intensity serves as a visual indication of the frequency of accurate and inaccurate predictions.

4.6 DECISION TREE CLASSIFIER

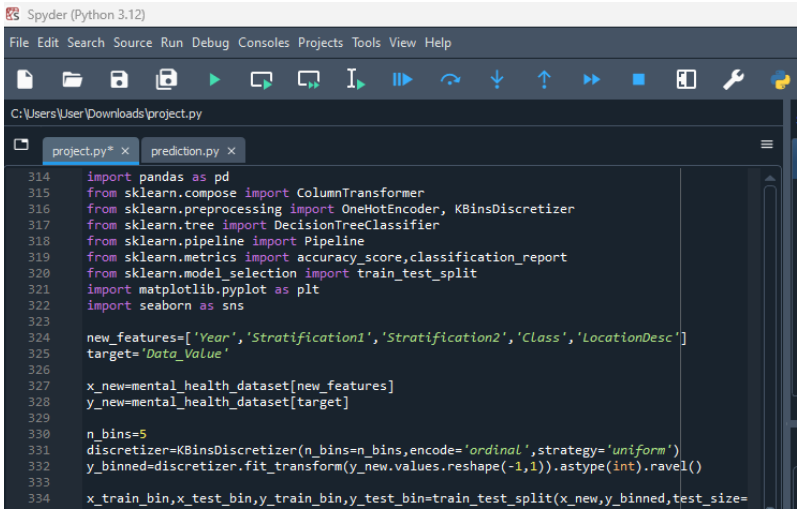


Figure 31:Decision Tree Classifier model setup

To train a Decision Tree Classifier to predict mental health measures, this snippet of code prepares the data. For data manipulation, scikit-learn is imported first, followed by matplotlib and seaborn for data visualisation, and then pandas for machine learning operations. "Year," "Stratification1," "Stratification2," "Class," and "LocationDesc" are among the features that were chosen for modelling. They were taken from the mental_health_dataset, and the goal variable to be predicted was "Data_Value." KBinsDiscretizer is used by the code to bin the data into five ordinal categories, converting the continuous target variable into discrete categories that can be classified. This process turns the regression problem into a classification endeavour. Then, using train_test_split, the dataset is divided into training and testing sets, with 80% designated for training and 20% for testing. The reproducibility of the data split is ensured by the random_state=42, yielding consistent findings across multiple runs. Preprocessing is a crucial step in ensuring that the data is structured and formatted correctly for machine learning, which prepares the groundwork for model training and assessment.

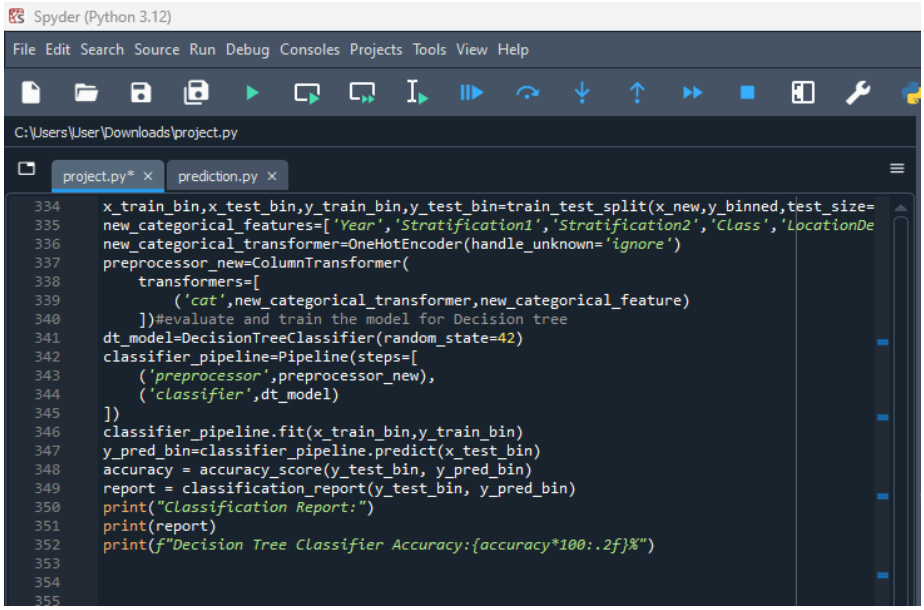


Figure 32:Encoding Categorical for Decision Tree Classifier

This code snippet provides a comprehensive approach that integrates data preprocessing and model training through a pipeline, enabling the training and evaluation of a Decision Tree Classifier to predict mental health parameters. In order to manage unknown categories and make sure they don't interfere with the model, it starts by establishing categorical features like "Year," "Stratification1," "Stratification2," "Class," and "LocationDesc." These features are then converted into a numerical format using OneHotEncoder. A ColumnTransformer is used to apply these changes, and the Decision Tree model and Pipeline are then combined. Before fitting the DecisionTreeClassifier to the training data, the Pipeline preprocesses the data to ensure that categorical variables are treated consistently. Random state = 42 is then used to ensure that the findings are repeatable. Precision, recall, and F1-score for each class are among the metrics that are included in the accuracy_score and classification_report, which offer a thorough evaluation of the model's performance based on its predictions on the test data.

```
Console 1/A ×
  _warn_prf(average, modifier, f {metric.capitalize()} is , len(result))
Classification Report:
              precision    recall  f1-score   support

           0       0.43      0.48      0.46       963
           1       0.41      0.39      0.40      1174
           2       0.14      0.11      0.12       197
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1

    accuracy                           0.40      2348
   macro avg       0.20      0.20      0.20      2348
weighted avg       0.39      0.40      0.40      2348

Decision Tree Classifier Accuracy:40.12%
```

Figure 33:Classification Report of Decision Tree Classifier

The model accurately identified the target class for just 40.12% of the test samples, according to the Decision Tree Classifier's classification report, which shows a mediocre overall accuracy of 40.12%. With somewhat better results for classes 0 and 1, where the F1-scores are 0.46 and 0.40, respectively, the model's variable performance is highlighted by the precision, recall, and F1-score for each class. But as seen by their F1-scores of 0.00, the model does not detect any examples of classes 3 or 4, and it performs poorly for class 2. Each class is treated equally by the macro average, but imbalances in class are taken into consideration by the weighted average.

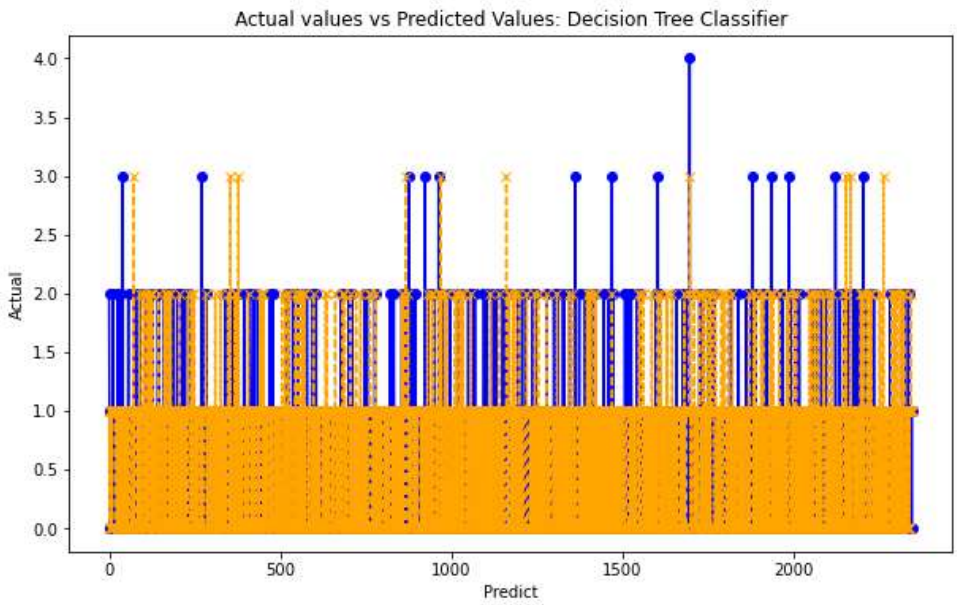


Figure 34:Actual vs Prediction of Decision Tree Classifier

The x-axis, which represents the predicted values and the y-axis, which displays the

actual target values, demonstrate the relationship between the predicted and real values from a Decision Tree Classifier. Orange points show anticipated values and blue points show actual values. To show the differences between an actual number and its associated anticipated value, each vertical line connects the two. The large vertical spread is a prominent characteristic that shows how frequently the model's predictions deviate from the actual results, particularly at higher real values. This pattern indicates that the model may have trouble correctly predicting specific classes or ranges of data, maybe as a result of imbalanced or overfitting data.
The model's frequent prediction of lower-value values, which could hide higher-value forecasts, indicates an imbalance in the dataset and a bias in the model's predictions.

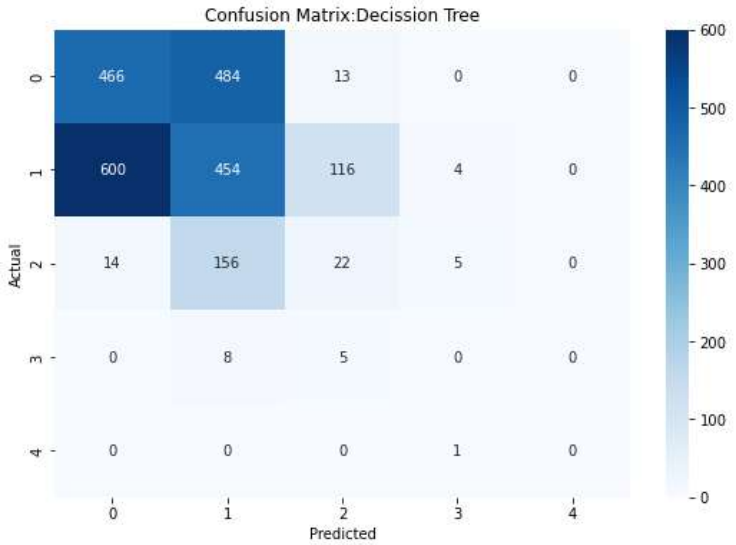


Figure 35: Confusion Matrix of Decision Tree Classifier

The Decision Tree model's confusion matrix compares the actual and predicted class labels to show how well the model performs in classification. Rows represent the actual classes, while columns represent the predicted classes. True positives, or correctly predicted classes, are indicated by diagonal cells, while misclassifications, or false positives and false negatives, are indicated by off-diagonal cells. With 466 correct predictions for class 0 and 454 for class 1, the matrix shows notable misclassifications between classes 0 and 1. Additionally, 484 cases of class 0 are misclassified as class 1 and 600 instances of class 1 are misclassified as class 0. This ambiguity shows that it may be challenging to tell these two groupings apart.

Additionally, the model has trouble with class 2, accurately predicting only 22 instances while misclassifying a large number of others as either class 1 or class 0. The rare

Deeper colours in the matrix denote larger counts, especially along the diagonal for classes 0 and 1. The colour intensity in the matrix represents the frequency of predictions.

4.7 KNN CLASSIFIER

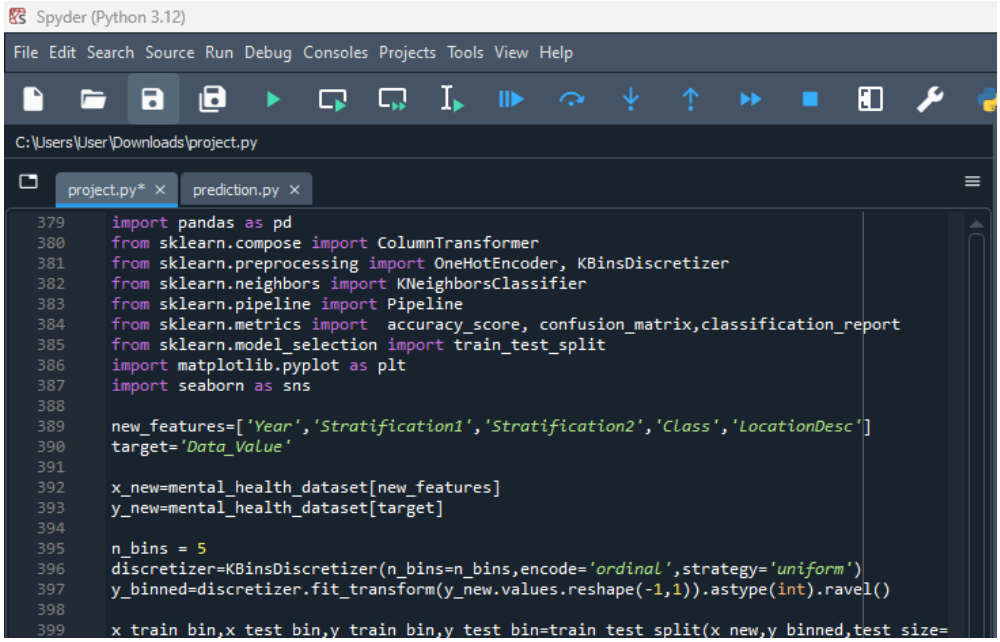


Figure 36:KNN model Setup

The code prepares data for predicting mental health metrics by training a K-Nearest Neighbours (KNN) classifier. First, the libraries required for machine learning, data manipulation, and visualisation are imported. With "Data_Value" as the target variable, the features chosen for the model are "Year," "Stratification1," "Stratification2," "Class," and "LocationDesc." To create the x_new and y_new datasets, respectively, these features and the target are taken out of the mental_health_dataset. The continuous Data_Value is transformed into a classification-friendly format by binning it into regularly spaced categories by KBinsDiscretizer, which discretises the target variable into five ordinal categories. Instead of projected continuous values, this transformation enables the model to categorise data into discrete groups. To ensure that the model's performance can be

tested on untested data, the data is then divided into training and testing sets, with 80% of the data utilised for training and 20% put aside for testing. Because random_state=42 is used, the data split is guaranteed to be repeatable, resulting in consistent results between runs.

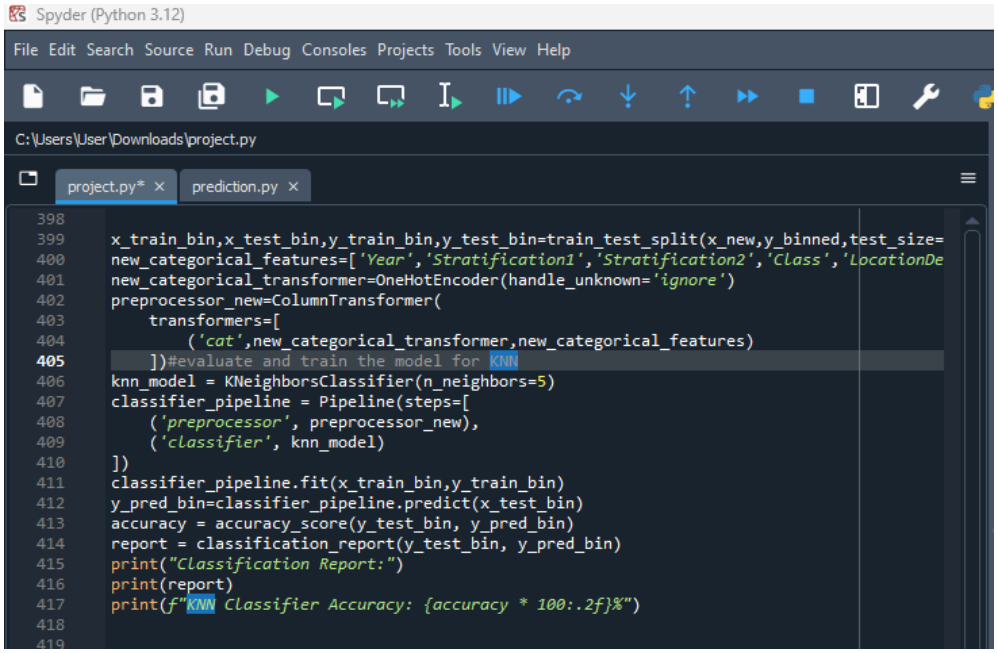


Figure 37:Encoding Categorical of KNN

Fig36:Encoding Categorical of KNN

To predict mental health measures, this code snippet configures and assesses a K-Nearest Neighbours (KNN) classifier. Data preprocessing and model training are integrated through a pipeline. The oneHotEncoder algorithm, which handles unknown categories by ignoring them and ensures robust model training, is used to transform categorical characteristics specified in the new_categorical_features list into a numerical format. Applying this transformation via a ColumnTransformer, the KNN model which has n_neighbors=5 configured to take into account the five closest neighbours for classification decisions is subsequently incorporated into a pipeline. After preprocessing the data, the pipeline fits the KNN model to the training set (x_train_bin, y_train_bin). Following training, the model's predictions on the test data (x_test_bin) are analysed using classification_report, which offers a thorough analysis of precision, recall, and F1-score for each class, and accuracy_score, which gauges the percentage of accurate predictions. The model's overall performance is

indicated by the final accuracy, which is printed. The classification report provides information about the model's ability to distinguish between various classes, emphasising both its strong points and potential areas for improvement.

```
Console 1/A ×
  warnings.warn(
Classification Report:
              precision    recall  f1-score   support

           0       0.51      0.54      0.53       963
           1       0.53      0.57      0.55      1174
           2       0.19      0.07      0.10       197
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1

    accuracy                           0.51      2348
   macro avg       0.25      0.24      0.24      2348
weighted avg       0.49      0.51      0.50      2348

KNN Classifier Accuracy: 51.19%
```

Figure 38:Classification Report of KNN

The K-Nearest Neighbours (KNN) classifier has a reasonable overall accuracy of 51.19%, which means that slightly more than half of its predictions on the test dataset were accurate, according to the classification report and accuracy score. With F1-scores of 0.53 and 0.55 for classes 0 and 1, respectively, the model achieves reasonable precision and recall, according to the report that breaks out the model's performance for each class. F1-scores of 0.00 for classes 3 and 4 indicate that the classifier completely fails to accurately classify any instances of these classes. In contrast, the classifier performs poorly for class 2, achieving an F1-score of just 0.10. As all classes are treated identically, the macro average of precision, recall, and F1-score is low at 0.24, indicating the model's difficulty with less frequent Class. Better results in the more common classes cause the weighted average, which takes into account the number of occurrences in each class, to marginally improve to 0.50.

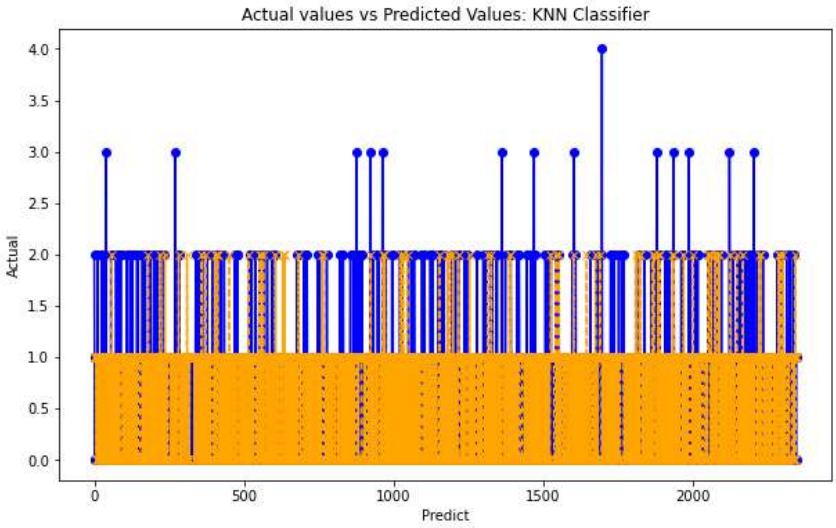


Figure 39:Actual vs Prediction of KNN

A K-Nearest Neighbours (KNN) classifier is used to predict values. The graph displays the relationship between the sample index and real values, with the actual values displayed on the y-axis and the predicted values represented on the x-axis. The expected values are represented by orange dots and lines, whereas the actual values are indicated by blue dots. The height of each vertical line that joins these dots represents the magnitude of the inaccuracy and represents a difference between an actual value and its related prediction. For ideal forecasts, there should be no lines or very few lines, indicating that the predictions and actual values match exactly. The model, however, is unable to forecast these cases with enough accuracy, since the graph shows a notable spread, especially for higher actual values. This trend points to potential problems, including model bias, which could be brought on by restrictions in the training data or class imbalances. Further evidence of the model's propensity to default to these more common classes while potentially ignoring less frequent ones comes from the dense clustering of lines at lower actual values.

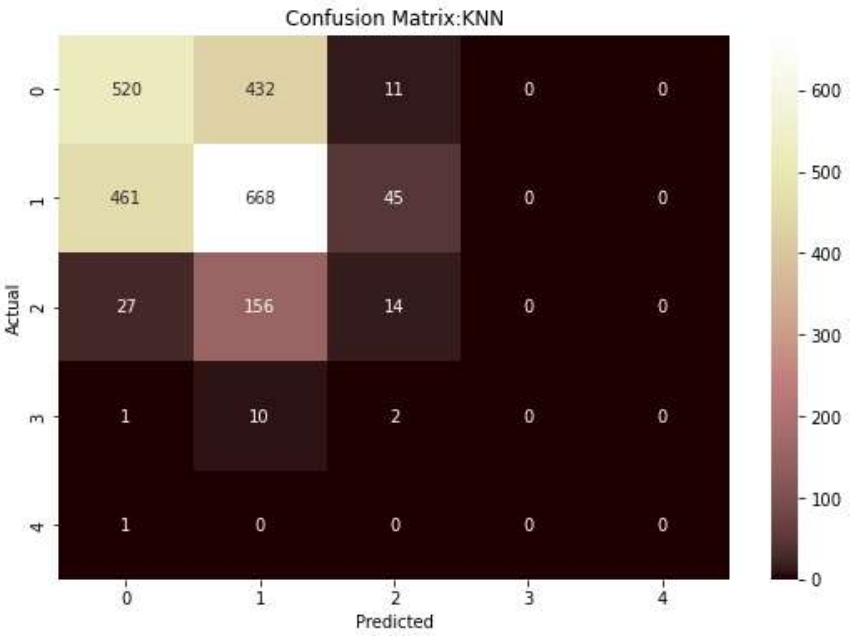


Figure 40:Confusion Matrix of KNN

The confusion matrix for the K-Nearest Neighbours (KNN) classifier illustrates how frequently actual classes are predicted correctly or incorrectly. Providing insight into the model's performance. Correct classifications are indicated by the diagonal cells, which correctly anticipate 668 cases of class 1 and 520 instances of class 0. However, there are notable misclassifications, particularly in the case of classes 0 and 1, where 461 cases of class 1 are incorrectly categorised as class 0 and 432 instances of class 0 are incorrectly forecasted as class 1. These misclassifications indicate that the model has difficulty telling these classes apart. Class 2 is particularly difficult to forecast; 156 cases were incorrectly classified as class 1, indicating a significant amount of difficulty. With the darker colours on the diagonal for classes 0 and 1, which suggest higher accuracy in these areas, the use of colour intensity highlights the distribution of predictions even more.

4.8 COMPARISON OF Accuracy, F1 Score, Recall and Precision

4.8.1 ACCURACY

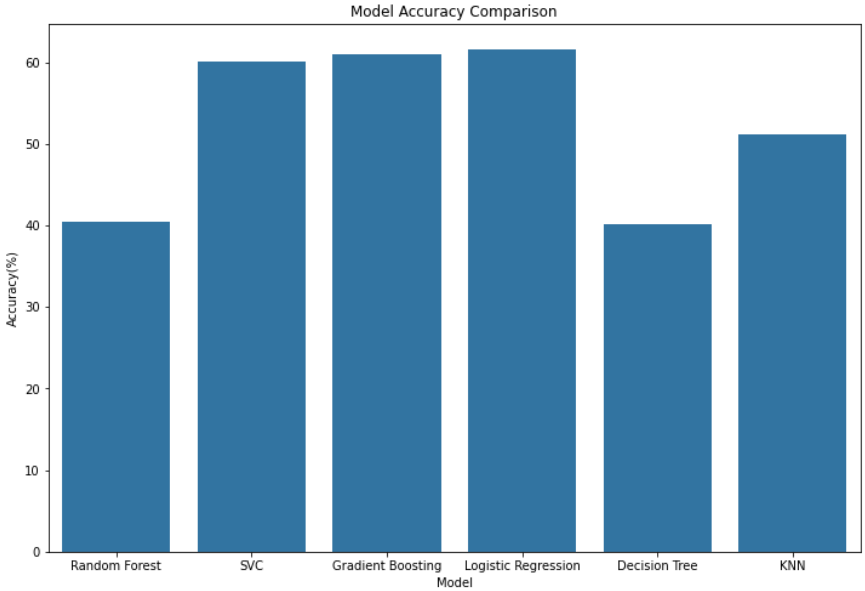


Figure 41:Model Comparison Accuracy

The accuracy of many machine learning models applied to a mental health dataset is compared in the bar chart, which focusses on the prediction of a discretised target variable (Data_Value). Random Forest, Gradient Boosting, Logistic Regression, Support Vector Classifier (SVC), Decision Tree, and KNN (K-Nearest Neighbours) are among the models that were assessed. A direct comparison of the models' efficacy is provided by each bar, which shows the proportion of properly predicted instances in the test set for each model. The y-axis in this graphical represents accuracy, a critical statistic in classification tasks, while the x-axis lists the models. The percentage of accurate outcomes among all instances investigated is known as accuracy. It provides a broad overview of each mod's performance.

Based on the findings, the models with the highest accuracy both above 60% are the Logistic Regression and Gradient Boosting models. This implies that these models are the most successful in predicting the mental health data in this particular project accurately out of all the models that were examined. SVC is a strong performer as well, as seen by its tight second position. However, the accuracy of the Random Forest and Decision Tree models is much lower; among the three, the Decision Tree

model performs the poorest, indicating a higher number of wrong predictions. The KNN model performs moderately well, outperforming both Random Forest and Decision Tree but falling short of the best.This analysis is especially important to the objectives of the research, which include investigating and assessing several machine learning methods for forecasting mental health indicators from ageing and Alzheimer's disease dataset. The goal of the study is to determine which predictive models are most useful for understanding mental health outcomes and maybe facilitating interventions. Precisely projecting these results can be essential for healthcare planning and interventions, since the goal variable in the dataset pertains to mental health.

4.8.2 Model Precision,F1 Score and Precision comparison

This chart's importance to the project lies in its capacity to offer a concise, contrasting summary of model performance. This holds special significance in the healthcare industry, as precise forecasts have the potential to improve patient outcomes and optimise resource utilisation.

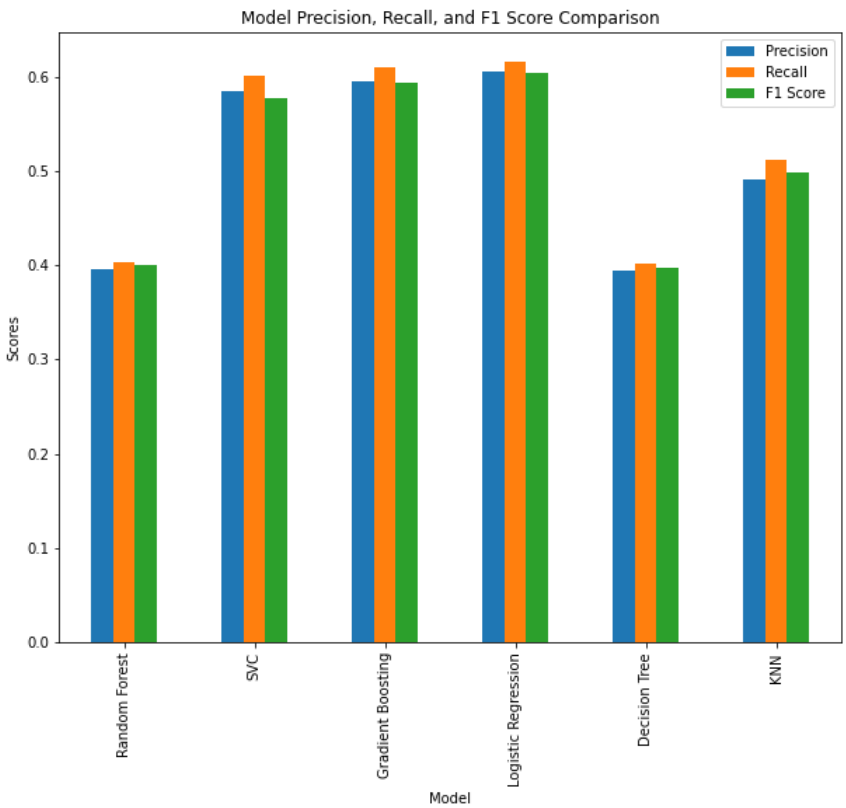
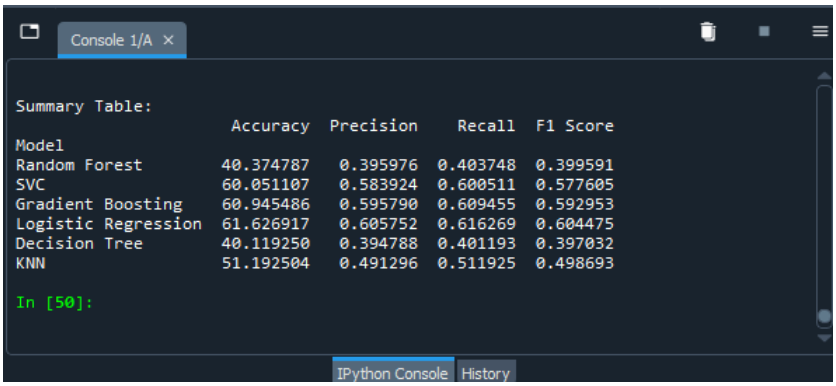


Figure 42:Model Metrics Comparison

The comparison of six machine learning models used to predict mental health metrics using three critical performance parameters precision, recall, and F1 score is shown in the bar chart. Random Forest, Gradient Boosting, Logistic Regression, Support Vector Classifier (SVC), Decision Tree, and K-nearest neighbours (KNN) are among the models. A distinct colour is used to represent each metric: green for F1 score, orange for recall, and blue for precision. Better performance is demonstrated by the SVC, Gradient Boosting, and Logistic Regression models. Metrics tend to cluster around the 0.6 thresholds, suggesting a well-balanced capacity to precisely predict positive cases (precision) and to record real positives (recall). These models also have strong F1 scores, which demonstrate how well they work overall to balance accuracy and recall.

Further, these models have strong F1 scores, which attests to their general efficacy in striking a balance between recall and precision. Lower scores for the Random Forest and Decision Tree models particularly in precision and recall indicate difficulties in correctly predicting and identifying positive cases. The KNN model exhibits a trade-off in these areas, as evidenced by its middling performance and more pronounced variance between recall and precision. The study depends on this analysis since it helps identify which models are more accurate at predicting mental health outcomes, which will help choose the optimal model for practical use. In healthcare settings, where both false positives and false negatives have substantial effects on patient care and budget allocation, the need for precision and memory is especially crucial.

4.9 SUMMARY TABLE



```
Summary Table:
                      Accuracy  Precision    Recall  F1 Score
Model
Random Forest        40.374787   0.395976  0.403748  0.399591
SVC                  60.051107   0.583924  0.600511  0.577605
Gradient Boosting    60.945486   0.595790  0.609455  0.592953
Logistic Regression  61.626917   0.605752  0.616269  0.604475
Decision Tree        40.119250   0.394788  0.401193  0.397032
KNN                  51.192504   0.491296  0.511925  0.498693

In [50]:
```

Figure 43:Model Performance Summary Table

Based on the above metrics, the Logistic Regression model is determined to be the top-performing model. When it came to predicting mental health metrics from the "Alzheimer's Disease and Healthy Ageing Data" dataset, our model outperformed all others with an accuracy rate of over 60%. A popular statistical technique for binary classification, logistic regression can also be applied to multiclass classification issues, as this project does. Balanced accuracy, recall, and F1 score demonstrate the model's effectiveness in properly recognising positive cases and reducing false positives and false negatives. By calculating the possibility that a certain input point belongs to a certain class, logistic regression calculates its workings. The simplicity and interpretability of this model are especially appreciated since it makes the decision-making process of the model easier to understand. The coefficients clearly show how each feature affects the forecast. Given its effectiveness in producing dependable and comprehensible predictions, Logistic Regression is a good fit for the dataset in question. It is particularly helpful in healthcare analytics, where deciphering the logic of the model is essential to making wise choices. When using the model in real-world scenarios where precise and fast forecasts are crucial, it's robustness and comparatively low computational cost also make it an attractive option.

5 PROJECT MANAGEMENT

To ensure that all activities are well-coordinated, resources are used properly, and goals are completed within the allotted timeframes, effective project management is essential to the successful completion of a research project. Several essential elements are included in the project management strategy for this research, which applies machine learning techniques to predict mental health metrics: planning, execution, monitoring, and assessment.

5.1 PROJECT PLANNING

Using the "Alzheimer's Disease and Healthy Ageing Data" dataset, the project's scope was precisely defined to concentrate on forecasting mental health measures. In doing so, the goals which included developing a model, evaluating performance, and preparing data were specified.

All available resources were made available, including the dataset, Python programming language, pertinent libraries (such scikit-learn, pandas, and matplotlib), and any hardware required for data processing and model training.

A comprehensive schedule was devised, incorporating benchmarks for significant phases like preparing data, choosing a model, training, and assessment. This made tracking development and project timeline adherence easier.

5.2 RISK MANAGEMENT

The successful identification, evaluation, and mitigation of any risks that could have an impact on the project's outcome were the main goals of effective risk management, which was an essential part of the research endeavour. Improper model training and prediction could result from data quality problems, such as missing values or inconsistent datasets. To lessen this, stringent cleaning and validation procedures as well as other data pretreatment measures were put in place. The possibility of computational constraints also existed, particularly in light of the substantial dataset and the requirement for thorough model training. A backup strategy that would guarantee sufficient processing power and memory availability was to use cloud-based computational resources. Additionally, there was a chance that the machine learning algorithms selected would not provide models with the accuracy or dependability that was needed. By choosing a wide variety of models,

this danger was reduced and made it possible to compare them and choose the best algorithm. To further improve model performance, cross-validation and hyperparameter tuning methods were used.

6 DISCUSSION

The "Alzheimer's Disease and Healthy Ageing Data" dataset is used in this study to apply machine learning algorithms for the purpose of predicting mental health metrics. Finding the best machine learning model to consistently predict these measures was the main goal of the work. These metrics are essential for early detection and treatment of mental health disorders, especially in older populations. Random Forest, Support Vector Classifier (SVC), Gradient Boosting Classifier, Logistic Regression, Decision Tree, and K-Nearest Neighbours (KNN) were among the well-known machine learning models that were thoroughly evaluated for this study. Accuracy, precision, recall, and F1 score were among the criteria used to evaluate each model's performance. These metrics offer a comprehensive view of the models' abilities, taking into account factors other than accuracy, such as how well the models differentiate between classes and manage erroneous positives and false negatives. As the model with the best balance of precision, recall, and F1 score, the logistic regression model came out on top. This implies that logistic regression is especially appropriate for this kind of data, probably because of its interpretability and simplicity, both of which are vital in medical environments. A key component of clinical decision-making is the ability of the model to give precise insights into the correlations between variables and outcomes, which can aid healthcare practitioners in understanding and having confidence in the predictions. Other models, such as SVC and Gradient Boosting, also demonstrated encouraging outcomes, suggesting their possible use in comparable predicting tasks. These models have a reputation for being reliable and for being able to identify detailed patterns in data, which is useful in healthcare analytics where complex linkages are frequently present in the data. However, lower accuracy and other metrics for the Decision Tree and Random Forest models indicate that they may not be able to handle the subtleties of this dataset as well, maybe because of overfitting or the intrinsic complexity of the data. To maintain the precision and dependability of the predictive models, the study also highlights the necessity of comprehensive data preprocessing, which includes encoding categorical variables and managing missing values. As a practical solution for managing continuous outcomes in a classification framework, the study's method of discretising the target variable is in line with real-world clinical circumstances where outcomes are frequently categorised.

7 CONCLUSION

This study used the "Alzheimer's Disease and Healthy Ageing Data" dataset to investigate and assess the efficacy of different machine learning models in predicting mental health metrics. Six distinct machine learning methods were thoroughly analysed in the study: Random Forest, Decision Tree, Gradient Boosting Classifier, Support Vector Classifier (SVC), Logistic Regression, and K-Nearest Neighbours (KNN). The efficacy of each model in forecasting mental health outcomes was evaluated using a range of performance indicators, such as accuracy, precision, recall, and F1 score.

7.1 ACHIEVEMENTS

The results demonstrate that, with a balanced performance across all measures, **Logistic Regression** proved to be the most successful model in this situation. This model is especially useful in healthcare situations where comprehending the model's decision-making process is essential. Its success can be ascribed to its simplicity, robustness, and interpretability. A dependable option for forecasting mental health measurements, logistic regression helps medical practitioners make well-informed decisions by offering lucid and practical insights into the data. Other models, such as SVC and Gradient Boosting, also showed good prediction ability, suggesting their possible use in related applications. On the other hand, models with substantially lower performance were Decision Tree and Random Forest, which might be because of overfitting issues or difficulties capturing the dataset's complexity. The findings highlight the crucial role that model selection plays in healthcare analytics, especially in delicate domains like mental health prediction. It emphasises how simple and easy-to-understand models like Logistic Regression are often more useful for real-world applications, even when sophisticated machine learning models can provide very high accuracy. To guarantee the validity and dependability of the models, the study also highlights the importance of thorough data preprocessing, which includes handling missing values and encoding categorical features. Overall, by offering light on the use of machine learning models for mental health prediction, this research advances the field of healthcare analytics. It implies that although a variety of models can be used for these kinds of tasks, Logistic Regression stands out for having a good balance between interpretability and accuracy.

7.2 FUTURE WORKS

Future studies on the use of machine learning models to forecast mental health measures may concentrate on several important topics. The accuracy and robustness of the model may be improved by incorporating data from other sources, such as longitudinal studies and more thorough demographic data. Sophisticated feature engineering methods, such as dimensionality reduction and interaction terms, can reveal hidden patterns in the data. Investigating ensemble approaches may help to improve performance by utilising the advantages of many models. Clinical applications must improve the interpretability and explainability of models using techniques such as SHAP values or LIME since this fosters professional trust and ease of use. It is important to address class imbalance in healthcare settings by utilising strategies such as SMOTE to enhance forecasts for under-represented outcomes. To implement these models in clinical practice, it is imperative to create personalised prediction models based on the unique characteristics of each patient and to concentrate on practical deployment, which includes user-friendly interfaces and real-world testing. Prioritising ethical and privacy issues is essential for guaranteeing data security and regulatory compliance. It is also necessary to set up procedures for ongoing model evaluation and upgrading to keep the model current and accurate throughout time. By working together, these initiatives will improve patient care and healthcare efficiency by improving the efficacy, dependability, and moral application of machine learning models in mental health prediction.

REFERENCES

1. *8 Types of Machine Learning Classification Algorithms*. (n.d.). Akkio. https://www.akkio.com/post/5-types-of-machine-learning-classification-algorithms
2. Abbas Saad Alatrany, Khan, W., Hussain, A., Hoshang Kolivand, & Dhiya Al-Jumeily. (2024). An explainable machine learning approach for Alzheimer's disease classification. *Scientific Reports*, *14*(1). https://doi.org/10.1038/s41598-024-51985-w
3. Aderghal, K. (2021). Classification of Multimodal MRI Images Using Deep Learning: Application to the Diagnosis of Alzheimer's Disease. In *Google Books*. https://books.google.co.uk/books/about/Classification_of_Multimodal_MRI_Images.html?id=x2Yf0AEACAAJ&redir_esc=y
4. Akhilesh Deep Arya, Verma, S., Prasun Chakarabarti, Chakrabarti, T., Elngar, A. A., Kamali, A.-M., & Nami, M. (2023). A systematic review on machine learning and deep learning techniques in the effective diagnosis of Alzheimer's disease. *Brain Informatics*, *10*(1). https://doi.org/10.1186/s40708-023-00195-7
5. Al Shehri, W. (2022). Alzheimer's disease diagnosis and classification using deep learning techniques. *PeerJ Computer Science*, *8*, e1177. https://doi.org/10.7717/peerj-cs.1177
6. Alroobaea, R., Mechti, S., Haoues, M., Rubaiee, S., Ahmed, A., Andejany, M., Bragazzi, N. L., Sharma, D. K., Kolla, B. P., & Sengan, S. (2021). *Alzheimer's Disease Early Detection Using Machine Learning Techniques*. https://doi.org/10.21203/rs.3.rs-624520/v1

7. Alzheimer's Association. (2023). 2023 alzheimer's disease facts and figures. *Alzheimer's & Dementia*, *19*(4), 1598–1695. https://doi.org/10.1002/alz.13016
8. Balakrishnan, N. B., Sreeja, P. S., & Panackal, J. J. (2022). Alzheimers Disease Diagnosis using Machine Learning: A Review. *International Journal of Engineering Trends and Technology*, *71*(3), 120–129. https://doi.org/10.14445/22315381/IJETT-V71I3P213
9. Bhushan, I., Department of Biotechnology, S. M. V. D. U., Kour, M., Kour, G., Gupta, S., Sharma, S., Yadav, A., & Department of Biotechnology, S. M. V. D. U. (2018). Alzheimer's disease: Causes & treatment – A review. *Annals of Biotechnology*, *1*(1). https://www.academia.edu/36194862/Alzheimers_disease_Causes_and_treatment_A_review
10. Bonci, A., Fiori, S., Higashi, H., Tanaka, T., & Verdini, F. (2021). An Introductory Tutorial on Brain–Computer Interfaces and Their Applications. *Electronics*, *10*(5), 560. https://doi.org/10.3390/electronics10050560
11. CDC. (2022, September 13). *Reducing Risk of Alzheimer's Disease*. Centers for Disease Control and Prevention. https://www.cdc.gov/aging/publications/features/reducing-risk-of-alzheimers-disease/index.htm
12. Dara, O. A., Lopez-Guede, J. M., Raheem, H. I., Rahebi, J., Zulueta, E., & Fernandez-Gamiz, U. (2023). Alzheimer's Disease Diagnosis Using Machine Learning: A Survey. *Applied Sciences*, *13*(14), 8298. https://doi.org/10.3390/app13148298
13. *Deep Learning-Based Alzheimer Disease Detection*. (n.d.). Springerprofessional.de. Retrieved July 31, 2024, from

https://www.springerprofessional.de/en/deep-learning-based-alzheimer-disease-detection/18397114

14. Diogo, V. S., Ferreira, H. A., & Prata, D. (2022). Early diagnosis of Alzheimer's disease using machine learning: a multi-diagnostic, generalizable approach. *Alzheimer's Research & Therapy*, *14*(1). https://doi.org/10.1186/s13195-022-01047-y
15. Gao, S., & Lima, D. (2022). A review of the application of deep learning in the detection of Alzheimer's disease. *International Journal of Cognitive Computing in Engineering*, *3*, 1–8. https://doi.org/10.1016/j.ijcce.2021.12.002
16. Islam, J., & Zhang, Y. (2018). Brain MRI analysis for Alzheimer's disease diagnosis using an ensemble system of deep convolutional neural networks. *Brain Informatics*, *5*(2). https://doi.org/10.1186/s40708-018-0080-3
17. Javeed, A., Dallora, A. L., Berglund, J. S., Ali, A., Ali, L., & Anderberg, P. (2023). Machine Learning for Dementia Prediction: A Systematic Review and Future Research Directions. *Journal of Medical Systems*, *47*(1). https://doi.org/10.1007/s10916-023-01906-7
18. Kanade, V. (2022, August 30). *What Is Machine Learning? Definition, Types, Applications, and Trends for 2022*. Spiceworks. https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-ml/
19. Kavitha, C., Mani, V., Srividhya, S. R., Khalaf, O. I., & Tavera Romero, C. A. (2022). Early-Stage Alzheimer's Disease Prediction Using Machine Learning Models. *Frontiers in Public Health*, *10*. https://doi.org/10.3389/fpubh.2022.853294
20. Khan, A., & Usman, M. (n.d.). *Early Diagnosis of Alzheimer's Disease using Machine Learning Techniques A Review Paper*.

21. *Learn Medical Diagnosis With Machine Learning: Advantages And Limitations*. (n.d.). Www.knowledgenile.com. Retrieved July 31, 2024, from https://www.knowledgenile.com/blogs/medical-diagnosis-with-machine-learning-advantages-and-limitations#:~:text=For%20Medical%20Diagnosis-
22. *Machine learning and medical diagnosis: disease detection*. (n.d.). Kili-Website. https://kili-technology.com/data-labeling/machine-learning/machine-learning-and-medical-diagnosis-an-introduction-to-how-ai-improves-disease-detection
23. Mirzaei, G., Adeli, A., & Adeli, H. (2016). Imaging and machine learning techniques for diagnosis of Alzheimer's disease. *Reviews in the Neurosciences*, *27*(8). https://doi.org/10.1515/revneuro-2016-0029
24. Ng, W. Y., Cheung, C. Y., Milea, D., & Ting, D. S. W. (2021). Artificial intelligence and machine learning for Alzheimer's disease: let's not forget about the retina. *British Journal of Ophthalmology*, *105*(5), 593–594. https://doi.org/10.1136/bjophthalmol-2020-318407
25. Shahbaz, M., Ali, S., Guergachi, A., Niazi, A., & Umer, A. (2019). Classification of Alzheimer's Disease using Machine Learning Techniques. *Proceedings of the 8th International Conference on Data Science, Technology and Applications*. https://doi.org/10.5220/0007949902960303
26. Tanveer, M., Richhariya, B., Khan, R. U., Rashid, A. H., Khanna, P., Prasad, M., & Lin, C. T. (2020). Machine Learning Techniques for the Diagnosis of Alzheimer's Disease. *ACM Transactions on Multimedia Computing, Communications, and Applications*, *16*(1s), 1–35. https://doi.org/10.1145/3344998

27. Taye, M. M. (2023). Understanding of Machine Learning with Deep Learning: Architectures, Workflow, Applications and Future Directions. *Computers*, *12*(5), 91. https://doi.org/10.3390/computers12050091
28. Trambaiolli, L. R., Lorena, A. C., Fraga, F. J., Kanda, P. A. M., Anghinah, R., & Nitrini, R. (2011). Improving Alzheimer's Disease Diagnosis with Machine Learning Techniques. *Clinical EEG and Neuroscience*, *42*(3), 160–165. https://doi.org/10.1177/155005941104200304
29. Trambaiolli, L., Lorena, A., Fraga, F., Kanda, P., Anghinah, R., & Nitrini, R. (2012). *Clinical EEG and Neuroscience Improving Alzheimer's Disease Diagnosis with Machine Learning Techniques On behalf of: EEG and Clinical Neuroscience Society can be found at: Clinical EEG and Neuroscience Additional services and information for*. https://doi.org/10.1177/155005941104200304
30. Uddin, K. M. M., Alam, M. J., Jannat-E-Anawar, Uddin, M. A., & Aryal, S. (2023). A Novel Approach Utilizing Machine Learning for the Early Diagnosis of Alzheimer's Disease. *Biomedical Materials & Devices*, *1*(2), 882–898. https://doi.org/10.1007/s44174-023-00078-9
31. Wikipedia Contributors. (2019, April 29). *Machine learning*. Wikipedia; Wikimedia Foundation. https://en.wikipedia.org/wiki/Machine_learning

APPENDIX : A

CODE AND OUTPUT

DATA PREPROCESSING

```
import pandas as pd
file_path="/Users/User/Downloads/Alzheimer_s_Disease_and_Healthy_Aging_Data (2).csv"
dataset=pd.read_csv(file_path)
print(dataset)
removing_columns=[
    'RowId', 'Question', 'Data_Value_Unit', 'DataValueTypeID', 'Data_Value_Type',
    'Data_Value_Alt', 'Data_Value_Footnote_Symbol', 'Data_Value_Footnote',
    'Geolocation', 'ClassID', 'TopicID', 'QuestionID', 'LocationID',
    'StratificationCategoryID3', 'StratificationID3', 'StratificationCategoryID2',
    'StratificationID2', 'LocationAbbr', 'Datasource', 'Topic', 'Low_Confidence_Limit',
    'High_Confidence_Limit', 'StratificationCategory1', 'StratificationCategory2'
]#removing the columns that are not required

data_preprocessed=dataset.drop(columns=removing_columns)
data_preprocessed['Year']=data_preprocessed['YearStart'].astype(str) +'-'+ data_preprocessed['YearEnd'].astype(str)#merge year
data_preprocessed=data_preprocessed.drop(columns=['YearStart','YearEnd'])
data_preprocessed.head()

cleaned_dataset=data_preprocessed.dropna()#removing missing values
cleaned_file_path='/Users/User/Downloads/Cleaned_Alzheimers_Disease_and_Healthy_Aging_Data.csv'
cleaned_dataset.to_csv(cleaned_file_path,index=False)
cleaned_dataset.head()

mental_health_dataset=cleaned_dataset[cleaned_dataset['Class']=='Mental Health']#dataset filter where 'class' is 'Mental Health'
filtered_file_path='/Users/User/Downloads//Mental_Health_Alzheimers_Disease_and_Healthy_Aging_Data.csv'
mental_health_dataset.to_csv(filtered_file_path,index=False)
mental_health_dataset.head()
```

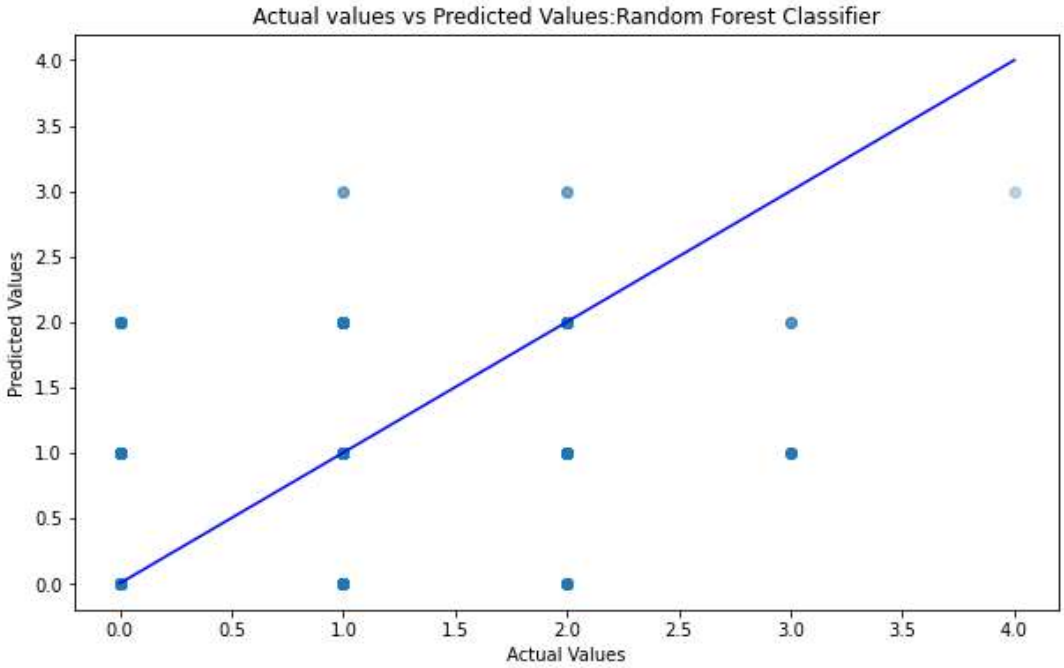
```
Console 1/A
284140    BRFSS~2019~2019~54~Q46~TOC10~AGE~RACE  ...                HIS
284141    BRFSS~2015~2015~56~Q02~TNC02~AGE~RACE  ...                NAA

[284142 rows x 31 columns]
Out[5]:
                      LocationDesc  ...       Year
3                         Maryland  ...  2022-2022
4                        Wisconsin  ...  2022-2022
6                         Oklahoma  ...  2022-2022
7                     Pennsylvania  ...  2022-2022
14  United States, DC & Territories  ...  2022-2022

[5 rows x 6 columns]

In [6]:
IPython Console  History
```

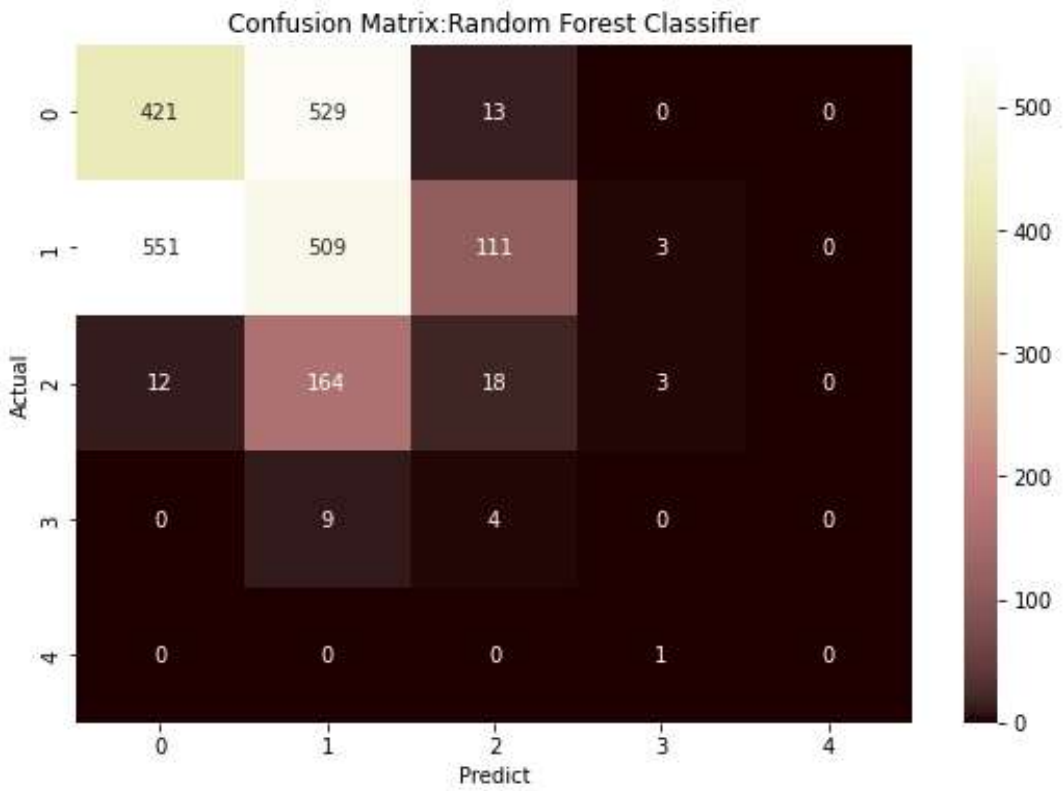
RANDOM FOREST CLASSIFIER

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, KBinsDiscretizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

new_features=['Year','Stratification1','Stratification2','Class','LocationDesc']
target='Data_Value'

x_new=mental_health_dataset[new_features]
y_new=mental_health_dataset[target]

n_bins=5
discretizer=KBinsDiscretizer(n_bins=n_bins,encode='ordinal',strategy='uniform')
y_binned=discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()#Data_value into categories

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=0.2,random_state=42)#splitting
```

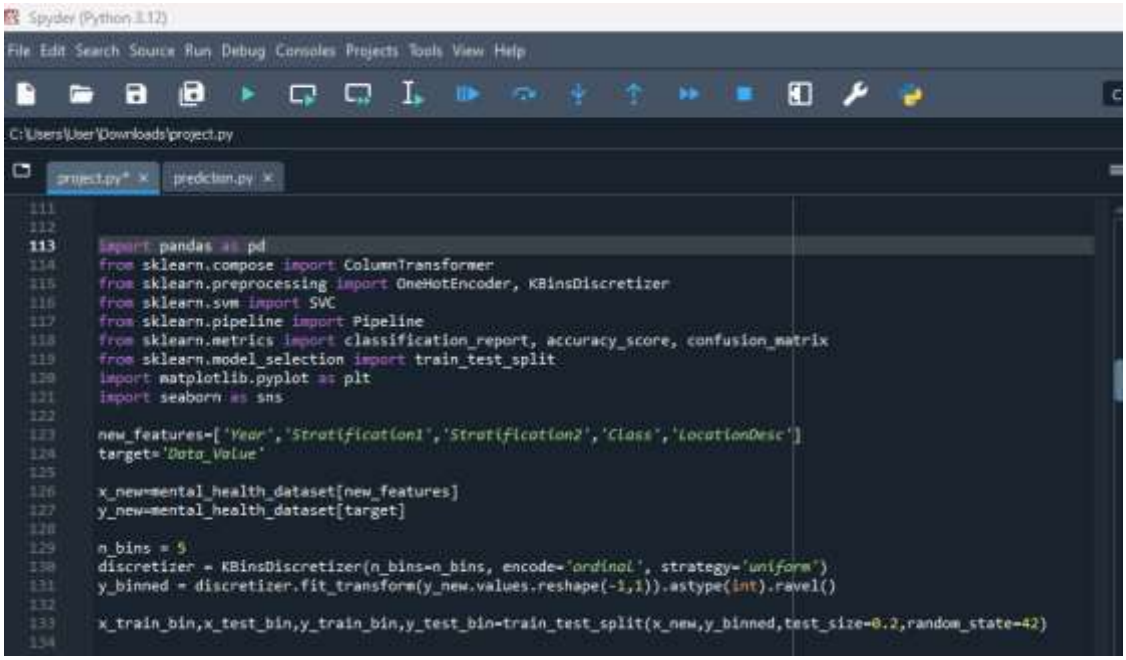
```python
y_binned=discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()#Data_value into categories

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=0.2,random_state=42)#splitting

new_categorical_features= ['Year','Stratification1','Stratification2','Class','LocationDesc']
new_categorical_transformer=OneHotEncoder(handle_unknown='ignore')
preprocessor_new=ColumnTransformer(
    transformers=[
        ('cat',new_categorical_transformer,new_categorical_features)
    ])
rf_model=RandomForestClassifier(random_state=42)
classifier_pipeline=Pipeline(steps=[
    ('preprocessor',preprocessor_new),
    ('classifier',rf_model)
])#evalute and train the model for random forest classifier
classifier_pipeline.fit(x_train_bin,y_train_bin)
y_pred_bin=classifier_pipeline.predict(x_test_bin)
accuracy=accuracy_score(y_test_bin,y_pred_bin)
report = classification_report(y_test_bin, y_pred_bin)
print("Classification Report:")
print(report)
print(f"Random Forest Classifier Accuracy:{accuracy*100:.2f}%")
```

```
Console 1/A ×

Classification Report:
              precision    recall  f1-score   support

           0       0.43      0.44      0.43       963
           1       0.42      0.43      0.43      1174
           2       0.12      0.09      0.10       197
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1

    accuracy                           0.40      2348
   macro avg       0.19      0.19      0.19      2348
weighted avg       0.40      0.40      0.40      2348

Random Forest Classifier Accuracy:40.37%
C:\Users\User\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1509:

IPython Console   History
```



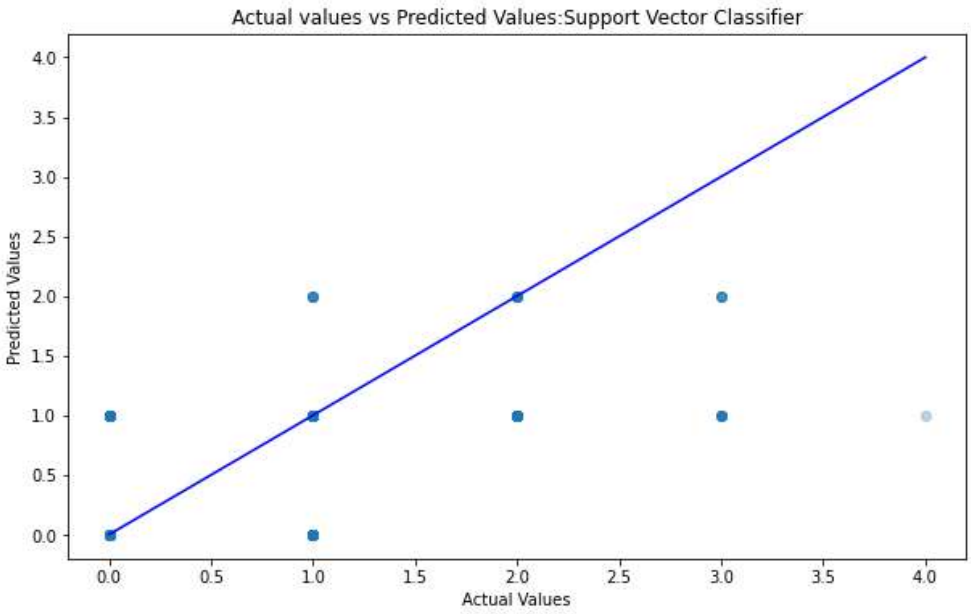


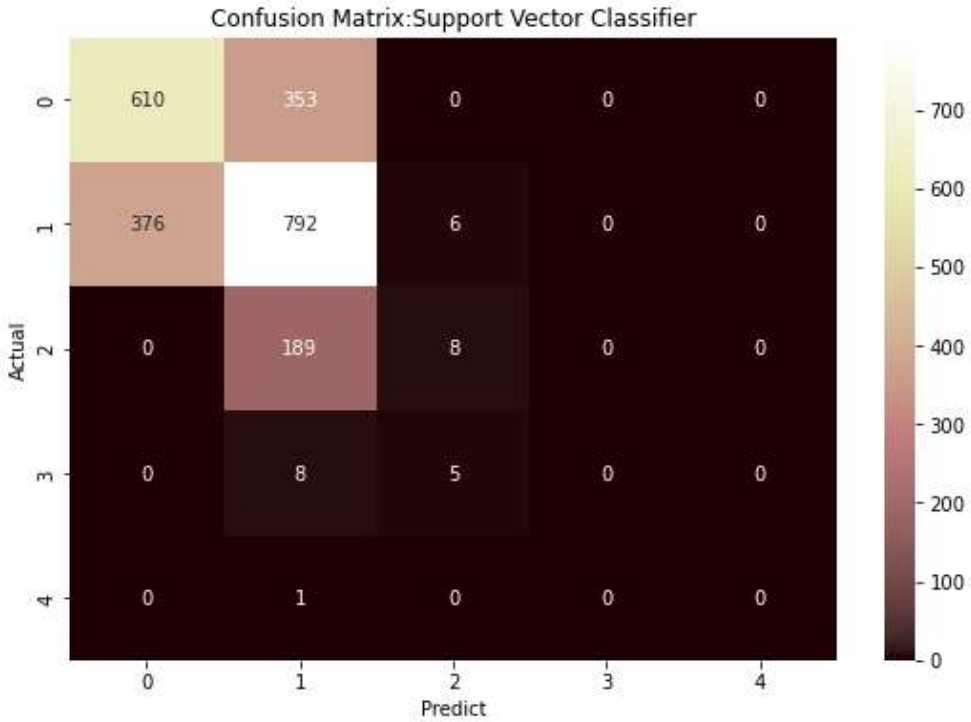
SUPPORT VECTOR CLASSIFIER



```python
x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=0.2,random_state=42)


new_categorical_features=['Year','Stratification1','Stratification2','Class','LocationDesc']
new_categorical_transformer=OneHotEncoder(handle_unknown='ignore')
preprocessor_new = ColumnTransformer(
    transformers=[
        ('cat',new_categorical_transformer,new_categorical_features)
    ])
svc_model = SVC(random_state=42)
classifier_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor_new),
    ('classifier', svc_model)
])#evaluate and train the model for support vector classifier
classifier_pipeline.fit(x_train_bin,y_train_bin)
y_pred_bin=classifier_pipeline.predict(x_test_bin)
accuracy=accuracy_score(y_test_bin, y_pred_bin)
print(f"Support Vector Classifier Accuracy:{accuracy*100:.2f}%")
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.62      0.63      0.63       963
           1       0.59      0.67      0.63      1174
           2       0.42      0.04      0.07       197
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1

    accuracy                           0.60      2348
   macro avg       0.33      0.27      0.27      2348
weighted avg       0.58      0.60      0.58      2348

Support Vector Classifier Accuracy:60.05%
```


Actual values vs Predicted Values:Support Vector Classifier
Predicted Values
Actual Values
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0


Confusion Matrix:Support Vector Classifier
Actual
Predict
610 353 0 0 0
376 792 6 0 0
0 189 8 0 0
0 8 5 0 0
0 1 0 0 0
0 1 2 3 4
700
600
500
400
300
200
100
0

LOGISTIC REGRESSION

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\User\Downloads\project.py

project.py* × prediction.py ×

```python

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, KBinsDiscretizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score,confusion_matrix
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns

new_features=['Year','Stratification1','Stratification2','Class','LocationDesc']
target='Data_Value'

x_new=mental_health_dataset[new_features]
y_new=mental_health_dataset[target]

n_bins=5
discretizer=KBinsDiscretizer(n_bins=n_bins,encode='ordinal',strategy='uniform')
y_binned=discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=
```

Spyder (Python 3.12)

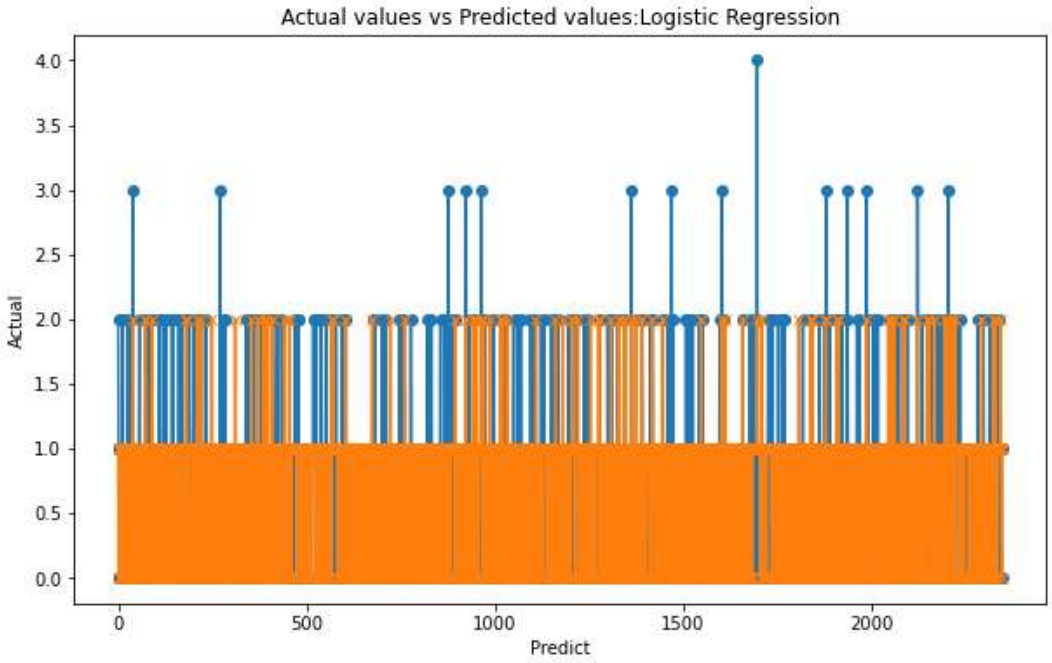
File Edit Search Source Run Debug Consoles Projects Tools View Help

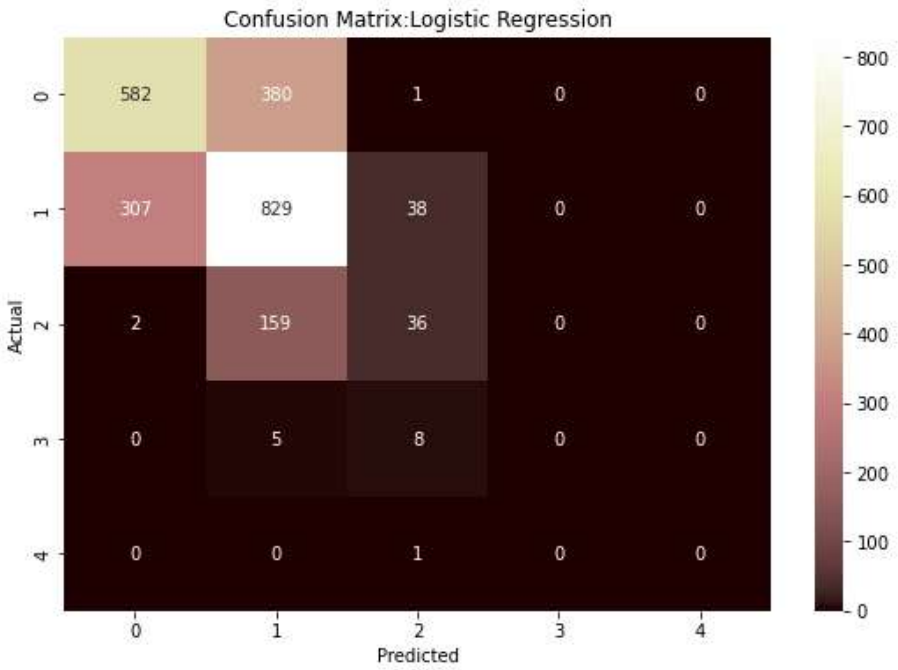
C:\Users\User\Downloads\project.py

project.py* × prediction.py ×

```python

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=
new_categorical_feature=['Year','Stratification1','Stratification2','Class','LocationDes
new_categorical_transformer=OneHotEncoder(handle_unknown='ignore')
preprocessor_new=ColumnTransformer(
    transformers=[
        ('cat',new_categorical_transformer,new_categorical_features)
    ])#evaluate and train the model for Logistic Regression
classifier_pipeline=Pipeline(steps=[
    ('preprocessor',preprocessor_new),
    ('classifier',LogisticRegression(max_iter=200,random_state=42))
])
classifier_pipeline.fit(x_train_bin,y_train_bin)
y_pred_bin=classifier_pipeline.predict(x_test_bin)
accuracy = accuracy_score(y_test_bin, y_pred_bin)
report = classification_report(y_test_bin, y_pred_bin)
print("Classification Report:")
print(report)
print(f"Logistic Regression Accuracy: {accuracy * 100:.2f}%")
```

```
Console 1/A ×
Classification Report:
              precision    recall  f1-score   support

           0       0.65      0.60      0.63       963
           1       0.60      0.71      0.65      1174
           2       0.43      0.18      0.26       197
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1

    accuracy                           0.62      2348
   macro avg       0.34      0.30      0.31      2348
weighted avg       0.61      0.62      0.60      2348

Logistic Regression Accuracy: 61.63%
```





CODE FOR GRADIENT BOOSTING CLASSIFIER

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\User\Downloads\project.py

project.py × prediction.py ×

```python

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, KBinsDiscretizer
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
import seaborn as sns
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt

new_features=['Year','Stratification1','Stratification2','Class','LocationDesc']
target='Data_Value'

x_new=mental_health_dataset[new_features]
y_new=mental_health_dataset[target]

n_bins=5
discretizer=KBinsDiscretizer(n_bins=n_bins,encode='ordinal',strategy='uniform')
y_binned=discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=
```

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

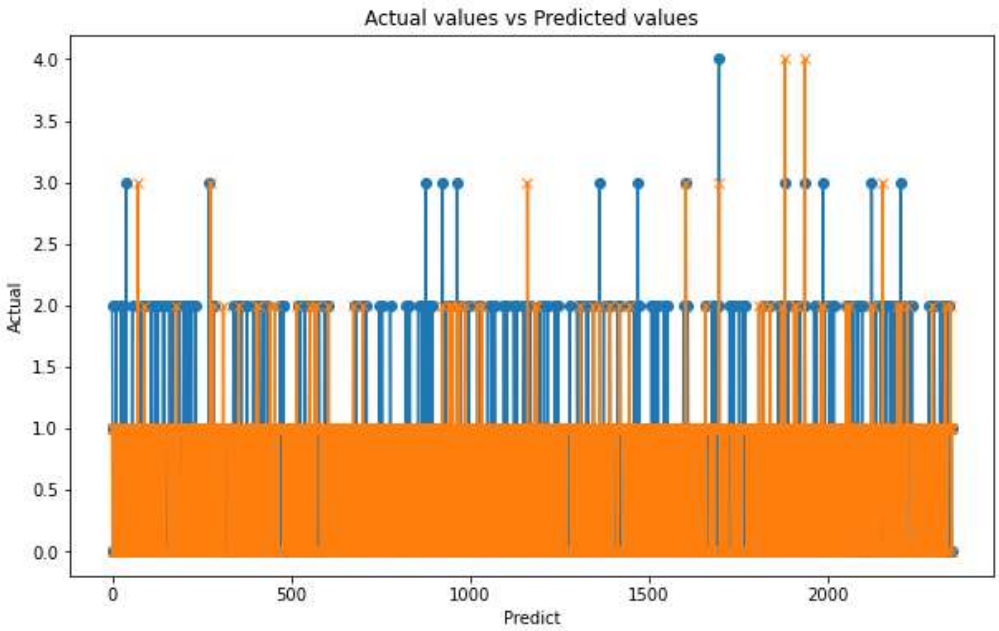
C:\Users\User\Downloads\project.py

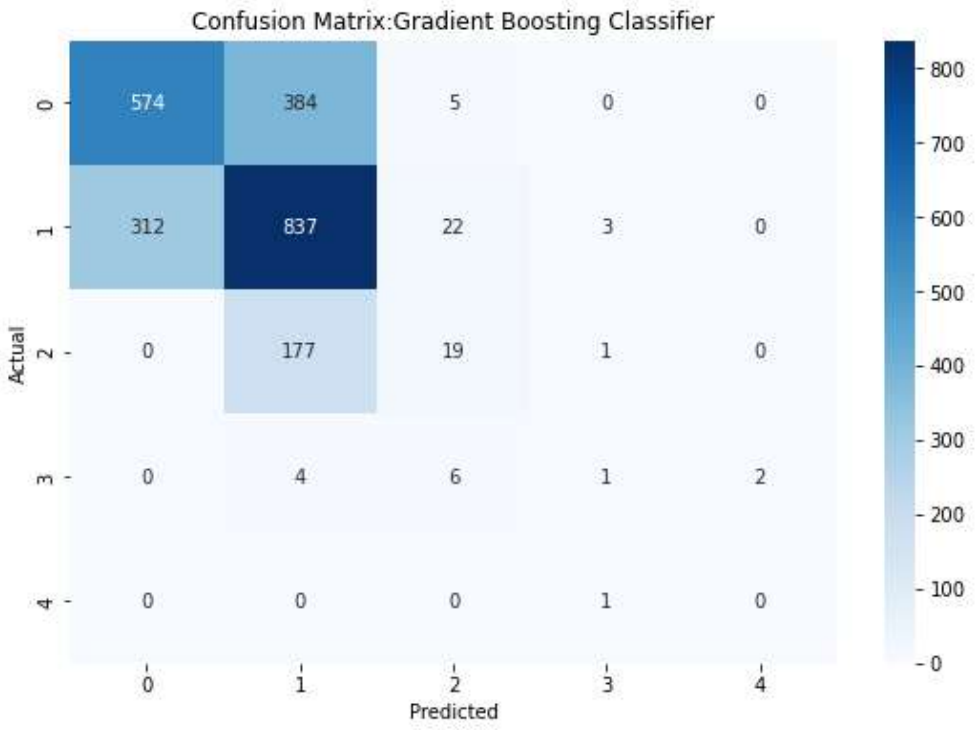
project.py* × prediction.py ×

```python
x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=

new_categorical_features=['Year','Stratification1','Stratification2','Class','LocationDe
new_categorical_transformer=OneHotEncoder(handle_unknown='ignore')
preprocessor_new=ColumnTransformer(
    transformers=[
        ('cat',new_categorical_transformer, new_categorical_features)
    ])#evaluate and tain the model for Gradient Boosting Classifier
classifier_pipeline=Pipeline(steps=[
    ('preprocessor',preprocessor_new),
    ('classifier',GradientBoostingClassifier(random_state=42))
])
classifier_pipeline.fit(x_train_bin, y_train_bin)
y_pred_bin=classifier_pipeline.predict(x_test_bin)
accuracy = accuracy_score(y_test_bin, y_pred_bin)
report = classification_report(y_test_bin, y_pred_bin)
print("Classification Report:")
print(report)
print(f"Gradient Boosting Classifier Accuracy:{accuracy*100:.2f}%")
```

Console 1/A ×

```
  warnings.warn(
Classification Report:
              precision    recall  f1-score   support

           0       0.65      0.60      0.62       963
           1       0.60      0.71      0.65      1174
           2       0.37      0.10      0.15       197
           3       0.17      0.08      0.11        13
           4       0.00      0.00      0.00         1

    accuracy                           0.61      2348
   macro avg       0.36      0.30      0.31      2348
weighted avg       0.60      0.61      0.59      2348

Gradient Boosting Classifier Accuracy:60.95%
```





DECISION TREE CLASSIFIER

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\User\Downloads\project.py

project.py* × prediction.py ×

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, KBinsDiscretizer
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score,classification_report
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns

new_features=['Year','Stratification1','Stratification2','Class','LocationDesc']
target='Data_Value'

x_new=mental_health_dataset[new_features]
y_new=mental_health_dataset[target]

n_bins=5
discretizer=KBinsDiscretizer(n_bins=n_bins,encode='ordinal',strategy='uniform')
y_binned=discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=
```

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

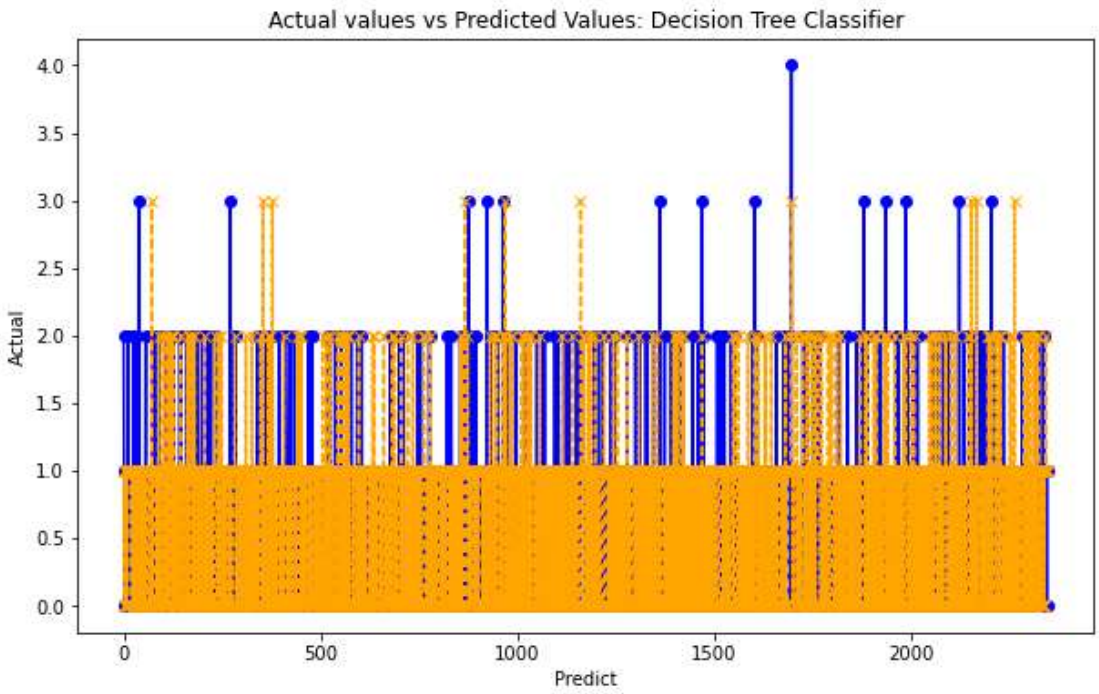
C:\Users\User\Downloads\project.py

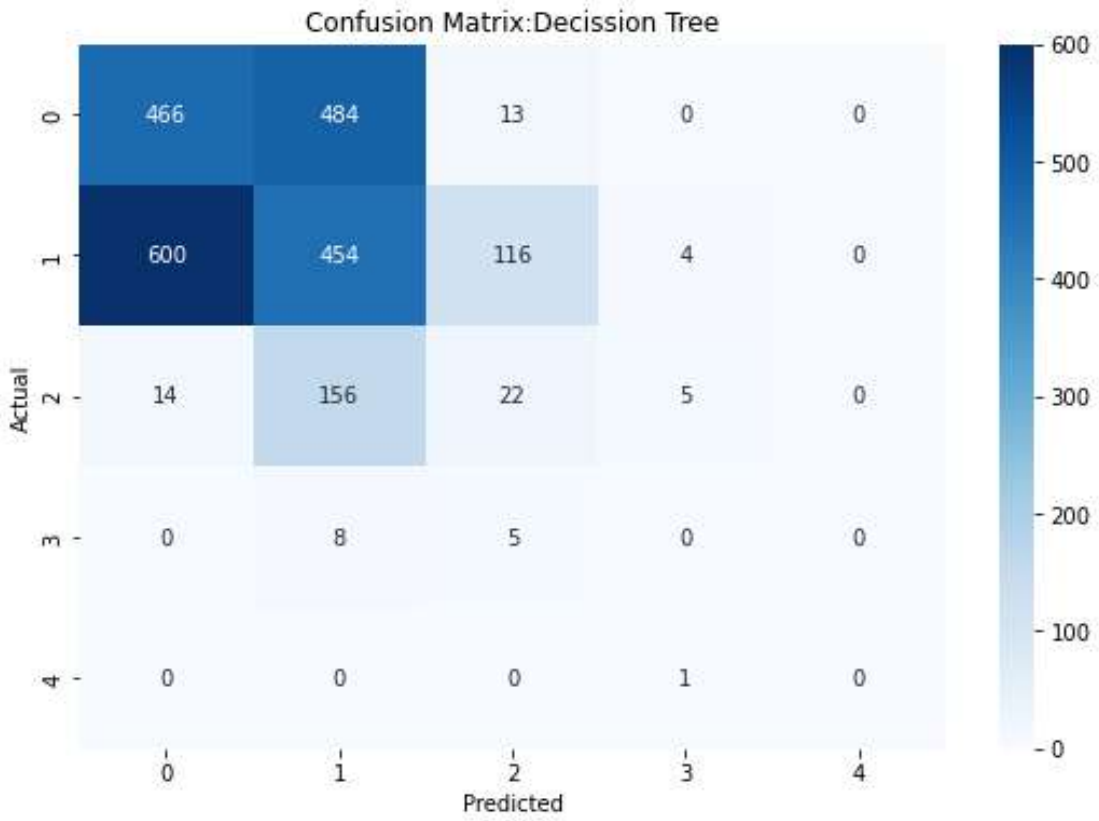
project.py* × prediction.py ×

```python
x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=
new_categorical_features=['Year','Stratification1','Stratification2','Class','LocationDe
new_categorical_transformer=OneHotEncoder(handle_unknown='ignore')
preprocessor_new=ColumnTransformer(
    transformers=[
        ('cat',new_categorical_transformer,new_categorical_feature)
    ])#evaluate and train the model for Decision tree
dt_model=DecisionTreeClassifier(random_state=42)
classifier_pipeline=Pipeline(steps=[
    ('preprocessor',preprocessor_new),
    ('classifier',dt_model)
])
classifier_pipeline.fit(x_train_bin,y_train_bin)
y_pred_bin=classifier_pipeline.predict(x_test_bin)
accuracy = accuracy_score(y_test_bin, y_pred_bin)
report = classification_report(y_test_bin, y_pred_bin)
print("Classification Report:")
print(report)
print(f"Decision Tree Classifier Accuracy:{accuracy*100:.2f}%")
```

Console 1/A ×

```
  _warn_prf(average, modifier, f {metric.capitalize()} is , len(result))
Classification Report:
              precision    recall  f1-score   support

           0       0.43      0.48      0.46       963
           1       0.41      0.39      0.40      1174
           2       0.14      0.11      0.12       197
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1

    accuracy                           0.40      2348
   macro avg       0.20      0.20      0.20      2348
weighted avg       0.39      0.40      0.40      2348

Decision Tree Classifier Accuracy:40.12%
```


Actual values vs Predicted Values: Decision Tree Classifier
Actual
Predict
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
500
1000
1500
2000


Confusion Matrix:Decission Tree
Actual
Predicted
466 484 13 0 0
600 454 116 4 0
14 156 22 5 0
0 8 5 0 0
0 0 0 1 0
0 1 2 3 4
600
500
400
300
200
100
0

KNN

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\User\Downloads\project.py

project.py* × prediction.py ×

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, KBinsDiscretizer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import  accuracy_score, confusion_matrix,classification_report
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns

new_features=['Year','Stratification1','Stratification2','Class','LocationDesc']
target='Data_Value'

x_new=mental_health_dataset[new_features]
y_new=mental_health_dataset[target]

n_bins = 5
discretizer=KBinsDiscretizer(n_bins=n_bins,encode='ordinal',strategy='uniform')
y_binned=discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=
```

Spyder (Python 3.12)

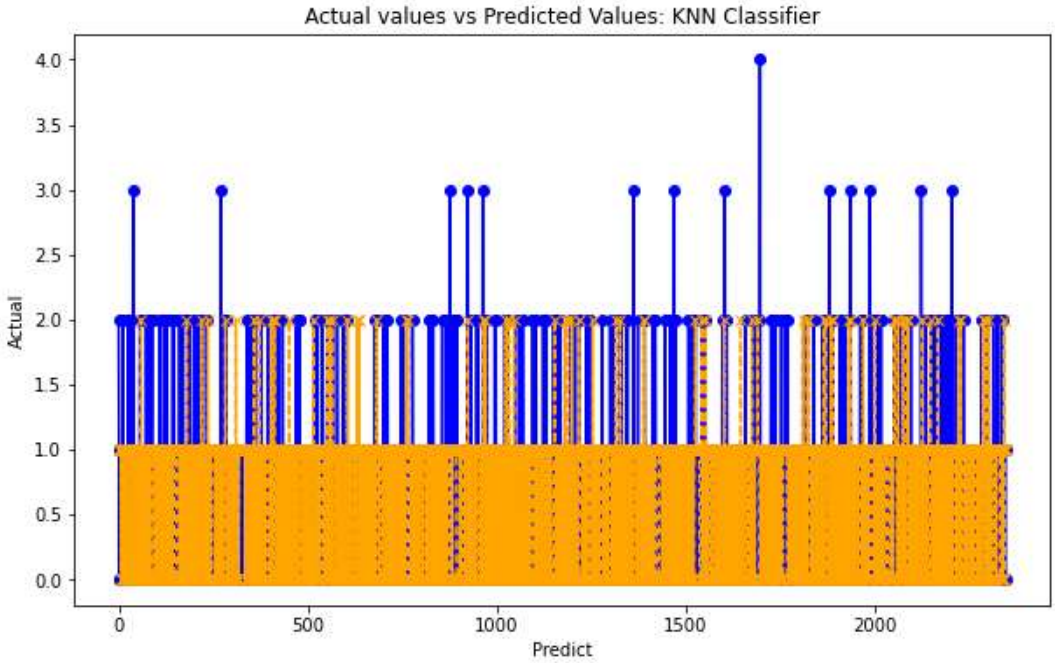
File Edit Search Source Run Debug Consoles Projects Tools View Help

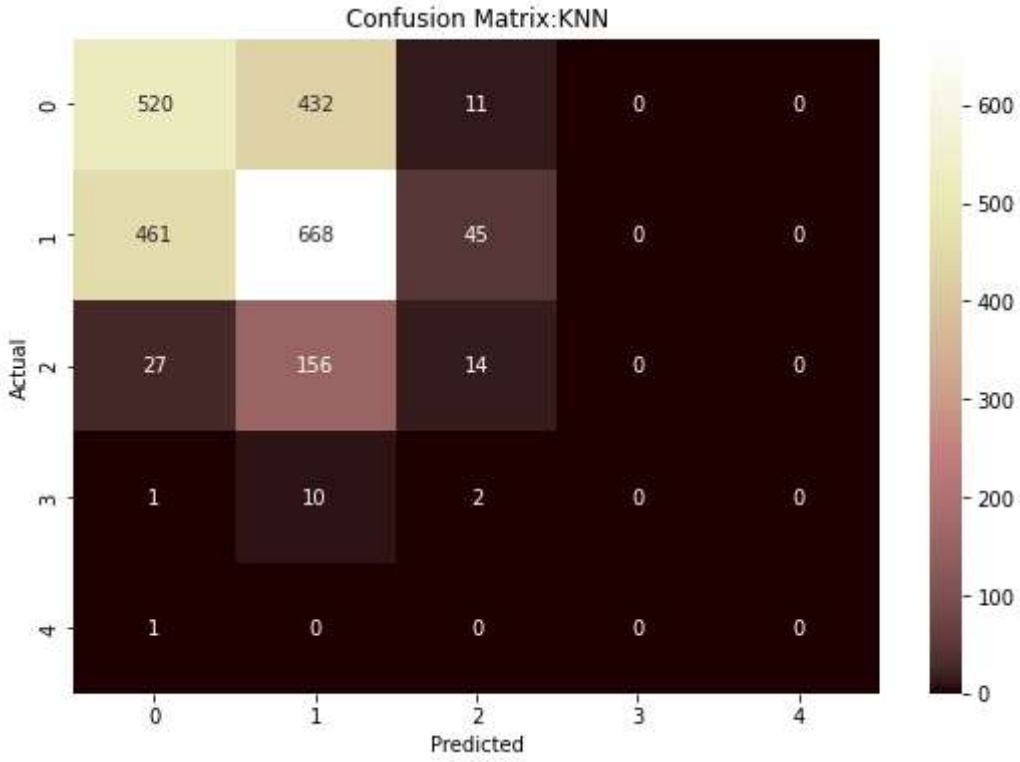
C:\Users\User\Downloads\project.py

project.py* × prediction.py ×

```python

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=
new_categorical_features=['Year','Stratification1','Stratification2','Class','LocationDe
new_categorical_transformer=OneHotEncoder(handle_unknown='ignore')
preprocessor_new=ColumnTransformer(
    transformers=[
        ('cat',new_categorical_transformer,new_categorical_features)
    ])#evaluate and train the model for KNN
knn_model = KNeighborsClassifier(n_neighbors=5)
classifier_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor_new),
    ('classifier', knn_model)
])
classifier_pipeline.fit(x_train_bin,y_train_bin)
y_pred_bin=classifier_pipeline.predict(x_test_bin)
accuracy = accuracy_score(y_test_bin, y_pred_bin)
report = classification_report(y_test_bin, y_pred_bin)
print("Classification Report:")
print(report)
print(f"KNN Classifier Accuracy: {accuracy * 100:.2f}%")
```

```
Console 1/A ×
  warnings.warn(
Classification Report:
              precision    recall  f1-score   support

           0       0.51      0.54      0.53       963
           1       0.53      0.57      0.55      1174
           2       0.19      0.07      0.10       197
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1

    accuracy                           0.51      2348
   macro avg       0.25      0.24      0.24      2348
weighted avg       0.49      0.51      0.50      2348

KNN Classifier Accuracy: 51.19%
```




Confusion Matrix:KNN
Actual
Predicted
0
1
2
3
4
520
432
11
0
0
461
668
45
0
0
27
156
14
0
0
1
10
2
0
0
1
0
0
0
0
600
500
400
300
200
100
0

ACCURACY RECALL F1 SCORE AND PRECISION COMPARISON AND SUMMARY TABLE

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\User\Downloads\project.py

project.py predicton.py*

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, KBinsDiscretizer
from sklearn.ensemble import RandomforestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns

new_features=['Year','Stratification1','Stratification2','Class','LocationDesc']
target='Data_Value'

x_new=mental_health_dataset[new_features]
y_new=mental_health_dataset[target]


n_bins=5
discretizer=KBinsDiscretizer(n_bins=n_bins,encode='ordinal',strategy='uniform')
y_binned=discretizer.fit_transform(y_new.values.reshape(-1,1)).astype(int).ravel()

x_train_bin,x_test_bin,y_train_bin,y_test_bin=train_test_split(x_new,y_binned,test_size=0.2,random_sta

new_categorical_transformer=OneHotEncoder(handle_unknown='ignore')
preprocessor_new=ColumnTransformer(
    transformers=[
        ('cat',new_categorical_transformer,new_features)
    ])
```

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\User\Downloads\project.py

project.py predicton.py*

```python
classifiers={
    'Random Forest':RandomForestClassifier(random_state=42),
    'SVC':SVC(random_state=42),
    'Gradient Boosting':GradientBoostingClassifier(random_state=42),
    'Logistic Regression':LogisticRegression(max_iter=200, random_state=42),
    'Decision Tree':DecisionTreeClassifier(random_state=42),
    'KNN':KNeighborsClassifier(n_neighbors=5)
}

results = []
classification_reports={}
for name, model in classifiers.items():
    pipeline=Pipeline(steps=[
        ('preprocessor',preprocessor_new),
        ('classifier',model)
    ])
    pipeline.fit(x_train_bin, y_train_bin)
    y_pred_bin=pipeline.predict(x_test_bin)
    accuracy=accuracy_score(y_test_bin,y_pred_bin)
    report=classification_report(y_test_bin,y_pred_bin,output_dict=True)
    precision=report['weighted avg']['precision']
    recall=report['weighted avg']['recall']
    f1_score=report['weighted avg']['f1-score']
    results.append({'Model':name,'Accuracy':accuracy*100,'Precision':precision,'Recall':recall,'F1 Sco
    classification_reports[name]=classification_report(y_test_bin, y_pred_bin)
    print(f"{name} Accuracy:{accuracy*100:.2f}%")
    print(f"Precision:{precision:.2f},Recall:{recall:.2f},F1 Score:{f1_score:.2f}")

results_dtfr=pd.DataFrame(results)
print("Comparison of Model Metrics:")
print(results_dtfr)
```

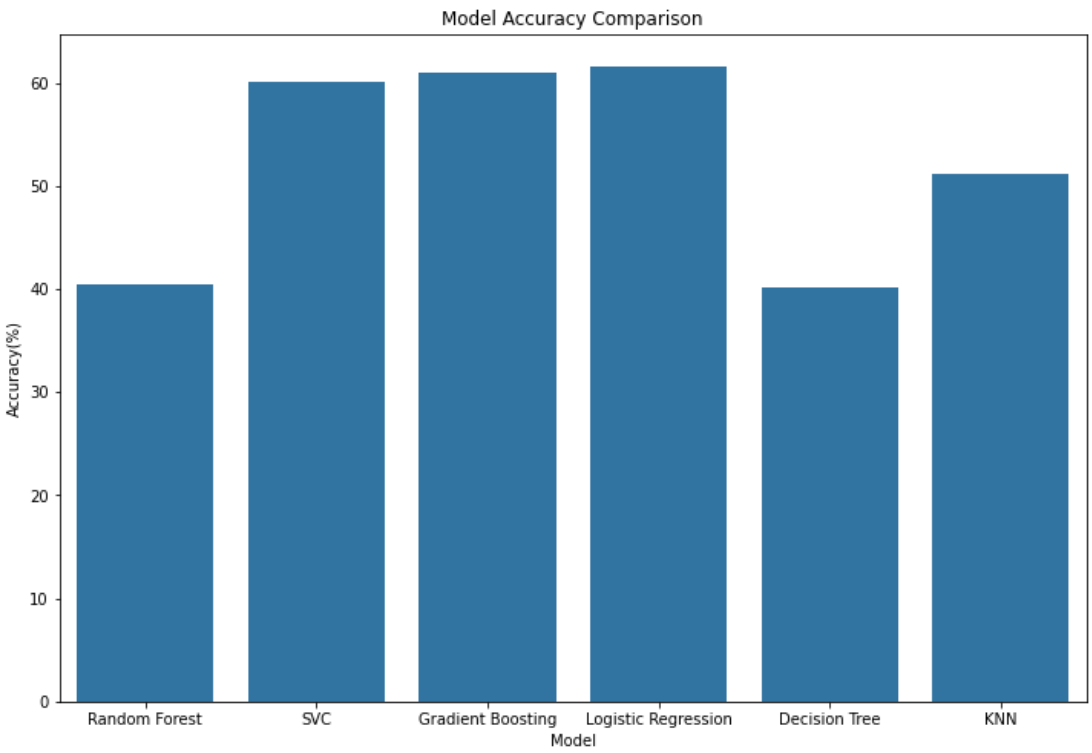
Spyder (Python 3.12)

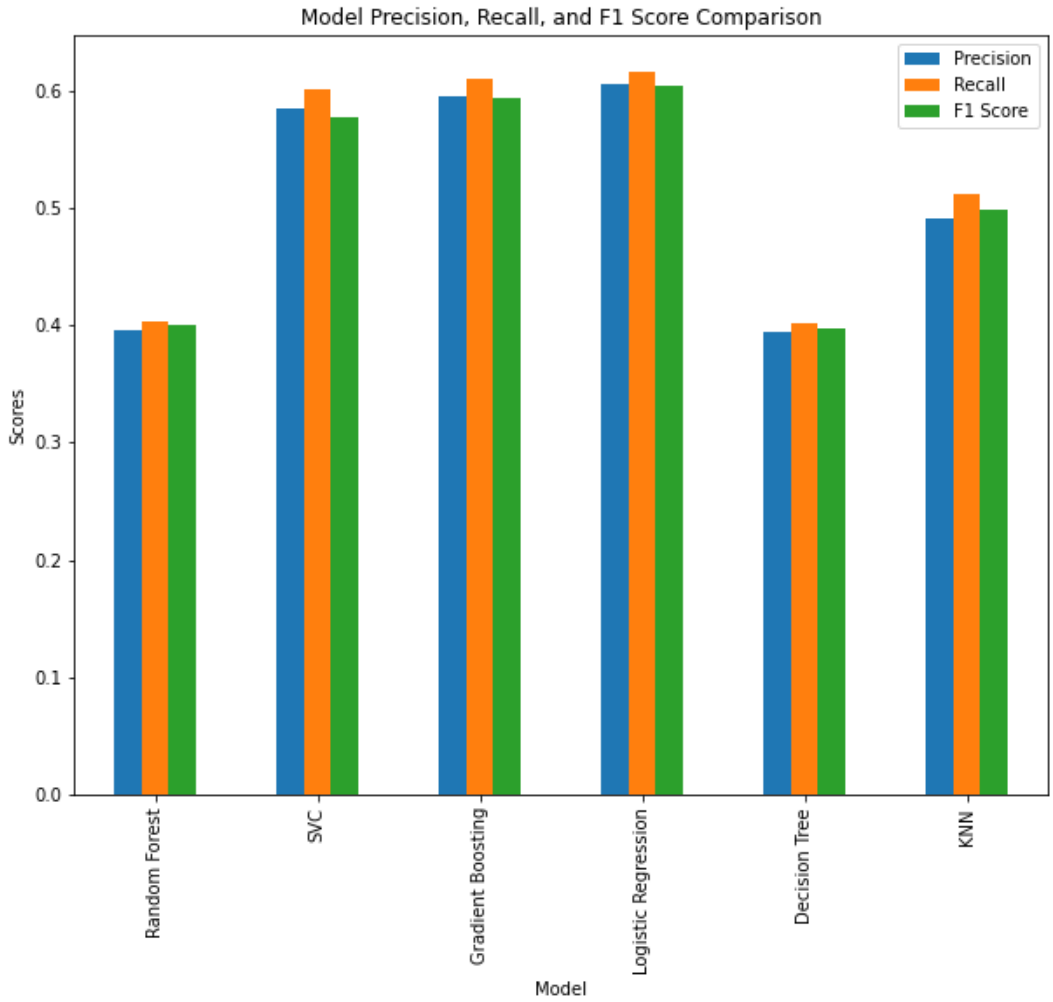
File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\User\Downloads\project.py

project.py × prediction.py* ×

```python
    results.append({'Model':name,'Accuracy':accuracy*100,'Precision':precision,'Recall':recall,'F1 Sco
    classification_reports[name]=classification_report(y_test_bin, y_pred_bin)
    print(f"{name} Accuracy:{accuracy*100:.2f}%")
    print(f"Precision:{precision:.2f},Recall:{recall:.2f},F1 Score:{f1_score:.2f}")

results_dtfr=pd.DataFrame(results)
print("Comparison of Model Metrics:")
print(results_dtfr)

plt.figure(figsize=(12,8))
sns.barplot(x='Model',y='Accuracy',data=results_dtfr)
plt.title('Model Accuracy Comparison')
plt.ylabel('Accuracy(%)')
plt.xlabel('Model')
plt.show()

results_dtfr.set_index('Model',inplace=True)
results_dtfr[['Precision','Recall','F1 Score']].plot(kind='bar',figsize=(10,8))
plt.title('Model Precision, Recall, and F1 Score Comparison')
plt.ylabel('Scores')
plt.xlabel('Model')
plt.show()

for model_name, report in classification_reports.items():
    print(f"Classification Report for {model_name}:\n{report}\n")

sum_table=results_dtfr
print("Summary Table:")
print(sum_table)
```






```
Console 1/A ×

Summary Table:
                     Accuracy  Precision    Recall  F1 Score
Model
Random Forest       40.374787   0.395976  0.403748  0.399591
SVC                 60.051107   0.583924  0.600511  0.577605
Gradient Boosting   60.945486   0.595790  0.609455  0.592953
Logistic Regression 61.626917   0.605752  0.616269  0.604475
Decision Tree       40.119250   0.394788  0.401193  0.397032
KNN                 51.192504   0.491296  0.511925  0.498693

In [50]:

IPython Console  History
```